

PythonNotes

WHAT IS PYTHON?

Python is a high-level, general-purpose programming language. That means you can use it to build almost anything—from websites and apps to data analysis tools and artificial intelligence.

It was created by Guido van Rossum and first released in 1991.

What is a High-Level Language?

A high-level language is a programming language that is easy for humans to read and write. It uses natural language elements (like English words) and hides the complex hardware details.

♦ Examples:

Python

Java

C++

JavaScript

Ruby

FEATURES:

Easy to learn

Portable (runs on different systems)

Closer to human language

You don't manage memory manually

Example (Python):

```
print("Hello, world!")
```

What is a Low-Level Language?

A low-level language is closer to machine language. It interacts directly with the computer's hardware, giving the programmer more control, but it's harder to write and understand.

♦ Two Types:

Machine Language (0s and 1s — binary)

Assembly Language (uses short codes like MOV, ADD)

 Example:
asm

```
MOV AH, 09h
MOV DX, OFFSET message
INT 21h
```

🌟 KEY FEATURES OF PYTHON

1.  Easy to Learn & Read
Python has a simple, English-like syntax.

Great for beginners.

```
python
print("Hello, world!")
```

2.  Interpreted Language
Python runs code line-by-line.

No need to compile before running.

Easier to test and debug.

3.  Dynamically Typed
You don't need to declare the data type of a variable.

```
python
x = 10    # x is an integer
x = "Ten" # Now x is a string
```

4.  OBJECT-ORIENTED PROGRAMMING (OOP)
Supports OOP concepts like classes, inheritance, polymorphism, etc.

```
python-----
```

```
class Car:
    def __init__(self, brand):
        self.brand = brand
```

```
my_car = Car("Toyota")
```

5. 📖 RICH STANDARD LIBRARY

Python comes with a large built-in library for file handling, math, regular expressions, networking, web development, etc.

```
python-----
```

```
import math  
print(math.sqrt(16))
```

6. 🌐 Cross-Platform

Python runs on Windows, macOS, Linux, etc.

Write once, run anywhere.

7. 🔗 OPEN SOURCE & FREE

Python is free to download, use, and share.

Community-developed, with regular updates.

8. ⚡ HIGH-LEVEL LANGUAGE

You don't need to manage memory or worry about low-level hardware details.

9. 🔌 Extensible & Embeddable

You can integrate Python with languages like C/C++.

Use Python inside other programs to add scripting.

10. 🧰 Wide Range of Applications

Web Development (Flask, Django)

Data Science (Pandas, NumPy)

Machine Learning (TensorFlow, scikit-learn)

Automation/Scripting

Game Development (Pygame)

Networking, Cybersecurity, etc.

 Bonus: Community Support
Huge community

Tons of tutorials, documentation, and libraries

Day 2

What is a Variable?

A **variable** is like a **container** or a **labeled box** that stores data.

Example:

```
A = 5
B = 10
a = b
number = 3589
Name = "john"
age = 25
Name = "uniq"
```

- `name` is a variable that stores the text `"Alice"`
- `age` is a variable that stores the number `25`

Think of it like:

Variable Name	Value
name	"Alice"
age	25

You can later **use** or **change** these variables in your program.

What is a Data Type?

A **data type** tells the computer **what kind of value** is being stored.

For example:

- Is it a **number**?
- Is it **text**?
- Is it **True/False**?
- Is it a **list** of things?

Each type has different **behaviors** and **uses**.

Common Data Types in Python

1. String (**str**)

Text or characters, always enclosed in quotes.

```
name = "Alice"
```

2. Integer (**int**)

Whole numbers (no decimal).

```
age = 25
```

3. Float (**float**)

Numbers with decimals.

```
price = 19.99
```

4. Boolean (**bool**)

True or False values.

0 and 1 's

```
is_online = True  
is_admin = False
```

5. List (**list**)

A collection of values, ordered and changeable.

```
colors = ["red", "green", "blue"]  
Name = [3,2223,12,12,1,321,3,12,321]
```

6. Dictionary (**dict**)

A collection of **key-value pairs**.

```
person = {"name": "Alice", "age": 25}
```

Key Points About Variables

- You create a variable using **=** (assignment).
- You don't need to tell Python what type it is—it figures it out automatically.
- Variable names **should be descriptive** (e.g., **price**, **user_name**, not **x**, **y** unless for math).
- Variables **can change** over time.

input() Function

The `input()` function lets you take user input from the keyboard. It always returns a **string**, even if the user types a number.

✓ Basic Syntax:python

```
variable_name = input("Your message here: ")
```

🧠 Example:

```
python
```

```
name = input("Enter your name: ")
```

```
print("Hello", name)
```

📘 What happens:

- Python shows: `Enter your name:`
- You type: `Aarav`
- It prints: `Hello Aarav`

⚠ Important Note:

Even if you type a number, Python treats it as a string:

```
python
```

```
age = input("Enter your age: ")
```

```
print(age + 1) # ❌ Error! You can't add a number to a string
```

To fix this, you need **type conversion**:

🔄 Type Conversion with `input()`

Use `int()` or `float()` to convert the input to a number:

✓ Convert to Integer:

```
age = int(input("Enter your age: "))
```

```
print(age + 1) # Works fine
```

M

✅ Convert to Float:

python

```
price = float(input("Enter the price: "))
```

```
print(price * 2)
```

Day 3

Arithmetic Operator

◆ 1. + (Addition Operator)

✅ What it does:

Adds two numbers.

🧠 Example:

python

```
x = 5
```

```
y = 3
```

```
result = x + y
```

```
print(result) # Output: 8
```

Explanation:

It adds $5 + 3$, which gives 8.

◆ 2. $-$ (Subtraction Operator)

What it does:

Subtracts the right-hand value from the left-hand value.

Example:

```
python
Code :
x = 5
y = 3
result = x - y
print(result) # Output: 2
```

Explanation:

$5 - 3$ equals 2.

◆ 3. $*$ (Multiplication Operator)

What it does:

Multiplies two numbers.

Example:

```
python
Code :
x = 5
y = 3
result = x * y
print(result) # Output: 15
```

Explanation:

5 * 3 equals 15.

◆ 4. / (Division Operator)

What it does:

Divides the left-hand number by the right-hand number and returns a **float** result.

Example:

python

Code :

```
x = 5
```

```
y = 2
```

```
result = x / y
```

```
print(result) # Output: 2.5
```

Explanation:

5 ÷ 2 equals 2.5. It returns a float even if the numbers divide evenly.

◆ 5. // (Floor Division Operator)

What it does:

Divides the left number by the right and returns **only the whole number part** (no decimals).

Example:

python

Code :

```
x = 5
```

```
y = 2
```

```
result = x // y
print(result) # Output: 2
```

Explanation:

$5 \div 2$ is 2.5, but floor division removes the decimal and returns 2.

◆ 6. % (Modulus Operator)

What it does:

Gives the **remainder** after division.

Example:

```
python
Code :
x = 5
y = 2
result = x % y
print(result) # Output: 1
```

Explanation:

$5 \div 2$ leaves a remainder of 1.

◆ 7. ** (Exponentiation Operator)

What it does:

Raises a number to the power of another.

Example:

```
python
Code :
```

```
x = 2
y = 3
result = x ** y
print(result) # Output: 8
```

Explanation:

`2 ** 3` means $2 * 2 * 2 = 8$.

2. Comparison Operators in Python

Comparison operators are used to compare two values. The result of any comparison is a **Boolean** value — either `True` or `False`.

Let's go through them one by one:

◆ 1. `==` (Equal To)

What it does:

Checks if two values are **equal**.

Example:

python

Code :

```
x = 5
y = 5
print(x == y) # Output: True
```

Explanation:

Since both `x` and `y` are 5, the expression returns `True`.

♦ 2. `!=` (Not Equal To)

✅ What it does:

Checks if two values are **not equal**.

🧠 Example:

python

Code :

```
x = 5
```

```
y = 3
```

```
print(x != y) # Output: True
```

📘 Explanation:

Because `5` is not equal to `3`, this returns `True`.

♦ 3. `>` (Greater Than)

✅ What it does:

Checks if the left value is **greater** than the right value.

🧠 Example:

python

Code :

```
x = 5
```

```
y = 3
```

```
print(x > y) # Output: True
```

Explanation:

5 > 3, so the comparison returns `True`.

◆ 4. < (Less Than)

What it does:

Checks if the left value is **less** than the right value.

Example:

python

Code :

```
x = 9
```

```
y = 10
```

```
print(x < y) # Output: True
```

Explanation:

Since 5 is less than 10, it evaluates to `True`.

◆ 5. >= (Greater Than or Equal To)

What it does:

Returns `True` if the left value is **greater than or equal to** the right value.

Example:

python

Code :

```
x = 5  
y = 5  
print(x >= y) # Output: True
```

Explanation:

5 >= 5 is True because they are equal.

◆ 6. <= (Less Than or Equal To)

What it does:

Returns True if the left value is **less than or equal to** the right value.

Example:

python

Code :

```
x = 3  
y = 5  
print(x <= y) # Output: True
```

Explanation:

3 <= 5 is True because 3 is less than 5.



3. Logical Operators in Python

Logical operators are used to combine multiple **conditions** (Boolean expressions). These are most commonly used in **if statements** and other **decision-making** structures.

◆ 1. **and** Operator



What it does:

Returns **True** only if both conditions are true.



Example:

python

Code :

```
x = 5  
  
print(x > 3 and x < 10) # Output: True
```



Explanation:

- `x > 3` is **True**
- `x < 10` is **True**
- Since both are **True**, the **and** operator returns **True**.



If one condition is false:

python

Code :

```
x = 5  
  
print(x > 3 and x > 10)  # Output: False
```

◆ 2. **or** Operator

✓ What it does:

Returns **True** if at least one condition is true.

🧠 Example:

python

Code :

```
x = 5  
  
print(x < 3 or x < 10)  # Output: True
```

📘 Explanation:

- `x < 3` is **False**
 - `x < 10` is **True**
 - Since one is **True**, the **or** operator returns **True**.
-

◆ 3. **not** Operator

✓ What it does:

Reverses the Boolean value:

- `not True` becomes `False`
- `not False` becomes `True`

Example:

python

Code :

```
x = 5
```

```
print(not (x > 3)) # Output: False
```

Explanation:

- `x > 3` is `True`
- `not True` becomes `False`

Quick Summary Table:

Expression	Result
<code>True and True</code>	<code>True</code>
<code>True and False</code>	<code>False</code>
<code>False or True</code>	<code>True</code>

`not False` `True`

`not True` `False`

4. Assignment Operators in Python

Assignment operators are used to **assign values to variables**. Some also combine an operation with the assignment — like `+=` adds and assigns in one step.

◆ 1. `=` (Simple Assignment)

What it does:

Assigns a value to a variable.

Example:

python

Code :

```
x = 10
```

```
print(x) # Output: 10
```

Explanation:

The value `10` is stored in variable `x`.

◆ 2. `+=` (Add and Assign)

✓ What it does:

Adds the right-hand value to the variable and assigns the result back.

🧠 Example:

python

Code :

```
x = 5  
  
x += 3  # Same as: x = x + 3  
  
print(x)  # Output: 8
```

```
Y = 5
```

```
Y += 3
```

◆ 3. -= (Subtract and Assign)

✓ What it does:

Subtracts the right-hand value from the variable and assigns the result back.

🧠 Example:

python

Code :

```
x = 5  
  
x -= 2  # Same as: x = x - 2  
  
print(x)  # Output: 3
```

◆ 4. ***=** (Multiply and Assign)

✅ What it does:

Multiplies the variable by the right-hand value and stores the result.

🧠 Example:

python

Code :

```
x = 4
x *= 2 # x = x * 2
print(x) # Output: 8
```

◆ 5. **/=** (Divide and Assign)

✅ What it does:

Divides the variable by the right-hand value and assigns the result (float).

🧠 Example:

python

Code :

```
x = 10
x /= 2 # x = x / 2
print(x) # Output: 5.0
```

◆ 6. //= (Floor Divide and Assign)

✓ What it does:

Divides and **removes decimals** (keeps only the integer part).

🧠 Example:

python

Code :

```
x = 10
x //= 3 # x = x // 3
print(x) # Output: 3
```

◆ 7. %= (Modulus and Assign)

✓ What it does:

Finds remainder and assigns it back.

🧠 Example:

python

Code :

```
x = 10
x %= 3 # x = x % 3
print(x) # Output: 1
```

◆ 8. ****=** (Power and Assign)

✓ What it does:

Raises the variable to the power of the right-hand value and assigns it back.

🧠 Example:

python

```
x = 2
x **= 3  # x = x ** 3
print(x)  # Output: 8
```

✓ 5. Bitwise Operators in Python (Intro)

Bitwise operators work at the **bit level** — they manipulate individual **binary digits (0s and 1s)** of integers. These are especially useful in low-level programming, graphics, cryptography, etc.

Before we begin, let's take a quick look at how numbers are represented in binary:

- 5 in binary = 0101
- 3 in binary = 0011

◆ 1. **&** (Bitwise AND)

✓ What it does:

Returns 1 if **both bits** are 1, otherwise 0.

Example:

python

```
x = 5 # 0101
y = 3 # 0011
print(x & y) # Output: 1
```

Explanation:

yaml

```
0101 (5)
& 0011 (3)
= 0001 → 1
```

◆ 2. | (Bitwise OR)

What it does:

Returns 1 if **either bit** is 1.

Example:

python

```
x = 5 # 0101
```

```
y = 3 # 0011
print(x | y) # Output: 7
```

Explanation:

yaml

```
0101
| 0011
= 0111 → 7
```

◆ 3. ^ (Bitwise XOR)

What it does:

Returns 1 if the bits are different.

Example:

python

```
x = 5 # 0101
y = 3 # 0011
print(x ^ y) # Output: 6
```

Explanation:

yaml

```
0101
^ 0011
= 0110 → 6
```

◆ 4. ~ (Bitwise NOT)

✓ What it does:

Inverts the bits: 1 becomes 0, and 0 becomes 1.
But in Python, it returns $-x - 1$.

🧠 Example:

python

```
x = 5
print(~x) # Output: -6
```

📘 Explanation:

$\sim 5 = -(5 + 1) = -6$

◆ 5. << (Left Shift)

✓ What it does:

Shifts bits to the **left**, adding 0s on the right.

🧠 Example:

python

```
x = 5 # 0101
print(x << 1) # Output: 10
```

📘 Explanation:

yaml

```
0101 << 1 = 1010 → 10
```

◆ 6. >> (Right Shift)

✓ What it does:

Shifts bits to the **right**, discarding the rightmost bits.

🧠 Example:

python

```
x = 5 # 0101
```

```
print(x >> 1) # Output: 2
```

Explanation:

yaml

```
0101 >> 1 = 0010 → 2
```

Python Operators: Full Summary Table

Category	Operator	Name / Meaning	Example	Result / Explanation
----------	----------	----------------	---------	----------------------

Arithmetic	+	Addition	5 + 3	8
	-	Subtraction	5 - 3	2
	*	Multiplication	5 * 3	15
	/	Division	5 / 2	2.5
	//	Floor Division	5 // 2	2
	%	Modulus (Remainder)	5 % 2	1
	**	Exponentiation	2 ** 3	8
Comparison	==	Equal to	5 == 5	True
	!=	Not equal to	5 != 3	True
	>	Greater than	5 > 3	True
	<	Less than	5 < 3	False

	<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	True
	<code><=</code>	Less than or equal to	<code>5 <= 3</code>	False
Logical	<code>and</code>	Logical AND	True and False	False
	<code>or</code>	Logical OR	True or False	True
	<code>not</code>	Logical NOT	not True	False
Assignment	<code>=</code>	Assign	<code>x = 5</code>	x is 5
	<code>+=</code>	Add and assign	<code>x += 3</code>	<code>x = x + 3</code>
	<code>-=</code>	Subtract and assign	<code>x -= 3</code>	<code>x = x - 3</code>
	<code>*=</code>	Multiply and assign	<code>x *= 3</code>	<code>x = x * 3</code>
	<code>/=</code>	Divide and assign	<code>x /= 3</code>	<code>x = x / 3</code>
	<code>//=</code>	Floor divide and assign	<code>x //= 3</code>	<code>x = x // 3</code>

	<code>%=</code>	Modulus and assign	<code>x %= 3</code>	<code>x = x % 3</code>
	<code>**=</code>	Exponentiate and assign	<code>x **= 3</code>	<code>x = x ** 3</code>
Bitwise	<code>&</code>	AND	<code>5 & 3</code>	<code>1</code> (0101 & 0011 = 0001)
	<code> </code>	OR	<code>5 3</code>	<code>7</code> (0101 0011 = 0111)
	<code>^</code>	XOR	<code>5 ^ 3</code>	<code>6</code> (0101 ^ 0011 = 0110)
	<code>~</code>	NOT	<code>~5</code>	<code>-6</code> (Inverts bits, returns <code>-x-1</code>)
	<code><<</code>	Left Shift	<code>5 << 1</code>	<code>10</code> (0101 becomes 1010)
	<code>>></code>	Right Shift	<code>5 >> 1</code>	<code>2</code> (0101 becomes 0010)

Python Conditionals

Conditional statements are used to **make decisions** in your code. Python uses `if`, `elif`, and `else` to execute code **based on conditions**.

◆ 1. **if** Statement

The **if** statement runs a block of code **only if the condition is True**.

Syntax:

```
python
Python
if condition:
    # code block
```

Example:

```
python
Python
x = 10
if x > 5:
    print("x is greater than 5")
```

Output: x is greater than 5

◆ 2. **else** Statement

The **else** block runs **if the if condition is False**.

Syntax:

```
python
Python
if condition:
    # if block
else:
    # else block
```

Example:

```
python
Python
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

Output: x is not greater than 5

◆ 3. **elif** Statement (else if)

The **elif** lets you check **multiple conditions** in order.

Syntax:

```
python
Python
if condition1:
    # print
elif condition2:
    # print
else:
    # print
```

Example:

```
python
Python
x = 5

if x > 10:
    print("x is greater than 10")
elif x == 5:
    print("x is exactly 5")
else:
    print("x is something else")
```

Output: `x is exactly 5`

◆ 4. Nested `if` Statements

You can put an `if` inside another `if` — this is called **nesting**.

✓ Example:

```
python
Python
x = 10
```

```
if x > 5:
    if x < 15:
        print("x is between 5 and 15")
```

Output: `x is between 5 and 15`

◆ 5. Indentation Rules



In Python, indentation is required to define code blocks.

- Use **consistent spacing** (usually 4 spaces per level)
- Missing or wrong indentation gives an **IndentationError**

✓ Correct:

```
python
Python
x = 7
if x > 5:
    print("x is big") # Indented properly
```

✗ Incorrect:

```
python
Python
x = 7
if x > 5:
print("x is big") # ❌ Error: not indented
```

Summary

Statement	Purpose
<code>if</code>	Checks a condition
<code>elif</code>	Checks another condition if previous was False
<code>else</code>	Runs when all previous conditions are False
Nested <code>if</code>	An <code>if</code> inside another <code>if</code>
Indentation	Required to mark code blocks

#####Daya 5#####

Looping Constructs: **while**, **for**, **range()**

while Loop

- **Purpose:** Repeats a block of code as long as a condition is **True**.

Syntax:

```
python
while condition:
    # code block
```

•

Example:

```
python
count = 1
while count <= 5:
    print(count)
    count += 1
```

-  Output: 1 2 3 4 5

for Loop

- **Purpose:** Iterates over a sequence (like a list, string, or range).

Syntax:

```
python
for item in sequence:
    # code block
```

•

Example:

```
python
for letter in "hello":
    print(letter)
```

-  Output: h e l l o

range() Function

- **Purpose:** Generates a sequence of numbers.

Syntax:

```
python  
range(start, stop, step)
```

-

Example:

```
python  
for i in range(1, 6):  
    print(i)
```

-  Output: 1 2 3 4 5



Loop Control Statements: **break, continue, else**



break

- **Purpose:** Exits the loop immediately.

Example:

```
python  
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)
```

-  Output: 1 2 3 4



continue

- **Purpose:** Skips the current iteration and moves to the next.

Example:

```
python  
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

-  Output: 1 2 4 5



else with Loops

- **Purpose:** Runs after the loop finishes normally (not via `break`).

Example:

```
python
for i in range(1, 4):
    print(i)
else:
    print("Loop completed")
```

-  Output: 1 2 3 Loop completed

With `break`:

```
python
for i in range(1, 4):
    if i == 2:
        break
    print(i)
else:
    print("Loop completed")
```

-  Output: 1 (no "Loop completed" because of `break`)

Day 6

1. What is a List?

- A **list** is an **ordered, mutable** (changeable) collection of items.
- It can store **different data types** together.
- Lists are written inside **square brackets** `[]`.

2. Creating Lists

```
# Empty list

empty_list = []
```

```
# List with integers

numbers = [1, 2, 3, 4]


# Mixed data types

mixed = [1, "hello", 3.14, True]


# Nested list (list inside a list)

nested = [1, [2, 3], ["a", "b"]]


print(numbers)

print(mixed)

print(nested)
```

✓ You can store **anything** in a list — even other lists.

3. Indexing

- Indexing means **accessing an element by its position**.
- **Index starts at 0** in Python.
- You can use **negative indexing** to access from the end.

```
fruits = ["apple", "banana", "cherry", "date"]
```

```
print(fruits[0])    # First element → apple
print(fruits[1])    # Third element → cherry
print(fruits[-1])   # Last element → date
print(fruits[-2])   # Second last → cherry
```

4. Slicing

- Slicing is **getting a portion of the list**.
- Syntax: `list[start:end:step]`
- `start` → index where slice begins (inclusive)
- `end` → index where slice stops (exclusive)
- `step` → skip interval

```
numbers = [10, 20, 30, 40, 50, 60]
```

```
print(numbers[1:4])    # From index 1 to 3 → [20, 30, 40]
print(numbers[:3])     # From start to index 2 → [10, 20, 30]
print(numbers[2:])     # From index 2 to end → [30, 40, 50, 60]
print(numbers[::2])    # Every 2nd element → [10, 30, 50]
print(numbers[::-1])   # Reverse list → [60, 50, 40, 30, 20, 10]
```

5. Common List Methods

Here are **important methods** with examples:

a) `append()`

- Adds **one** item at the **end** of the list.

```
fruits = ["apple", "banana"]  
fruits.append("cherry")  
print(fruits)  # ['apple', 'banana', 'cherry']
```

b) `extend()`

- Adds **multiple** items from another list.

```
fruits = ["apple", "banana"]  
fruits.extend(["cherry", "orange"])  
print(fruits)  # ['apple', 'banana', 'cherry', 'Orange']
```

c) `insert()`

- Inserts at a specific index.

```
fruits = ["apple", "banana"]  
fruits.insert(1, "mango")
```

```
print(fruits) # ['apple', 'mango', 'banana']
```

d) remove()

- Removes the **first occurrence** of a value.

```
fruits = ["apple", "banana", "cherry", "banana"]  
fruits.remove("banana")  
print(fruits) # ['apple', 'cherry', 'banana']
```

e) pop()

- Removes and returns an element by index (default last).

```
fruits = ["apple", "banana", "cherry"]  
last = fruits.pop()  
print(last)      # cherry  
print(fruits)    # ['apple', 'banana']
```

f) index()

- Returns **first index** of a value.

```
numbers = [10, 20, 30, 20]
```

```
print(numbers.index(20)) # 1
```

g) count()

- Counts how many times a value appears.

```
numbers = [10, 20, 20, 30, 20,20]
```

```
Abc = numbers.count(20)
```

```
print(abc) # 4
```

h) sort()

- Sorts the list **in place**.

```
numbers = [4, 1, 3, 2]
```

```
numbers.sort()
```

```
print(numbers) # [1, 2, 3, 4]
```

```
numbers.sort(reverse=True)
```

```
print(numbers) # [4, 3, 2, 1]
```

i) reverse()

- Reverses the list in place.

```
fruits = ["apple", "banana", "cherry"]  
  
fruits.reverse()  
  
print(fruits)  # ['cherry', 'banana', 'apple']
```

j) copy()

- Creates a shallow copy of the list.

```
original = [1, 2, 3]  
  
copy_list = original.copy()  
  
copy_list.append(4)  
  
print(original)  # [1, 2, 3]  
  
print(copy_list) # [1, 2, 3, 4]
```

Day 7

Tuples

1. What is a Tuple?

- A **tuple** is like a list, but it **cannot be changed** (immutable).
- It is **ordered** and can store different data types.

👉 Example:

```
fruits = ("apple", "banana", "cherry")  
print(fruits)
```

Output:

```
('apple', 'banana', 'cherry')
```

✅ Explanation: This tuple has 3 elements. The order is fixed, and you cannot change it later.

2. Immutability

```
numbers = (1, 2, 3)  
# Trying to change an element  
# numbers[0] = 99    # ❌ This gives error
```

✅ Explanation: Tuples **cannot be modified** once created. Unlike lists, you cannot add, remove, or replace items.

3. Tuple Unpacking

```
student = ("Alice", 20, "Math")  
name, age, subject = student  
print(name)    # Alice  
print(age)     # 20  
print(subject) # Math
```

✅ Explanation: Tuple values are directly assigned to variables in order. This is **tuple unpacking**.

4. When to Use Tuples

- When data should **not change**.

- To store **fixed records** (like coordinates, database rows).

👉 Example:

```
coordinates = (12.97, 77.59) # Latitude, Longitude
print("Latitude:", coordinates[0])
print("Longitude:", coordinates[1])
```

Output:

```
Latitude: 12.97
Longitude: 77.59
```

✅ Explanation: A city's coordinates never change → tuple is best.

Sets

1. What is a Set?

- A **set** is an unordered collection of **unique values**.
- Duplicates are removed automatically.

👉 Example:

```
nums = {1, 2, 2, 3, 4, 4}
print(nums)
```

Output:

```
{1, 2, 3, 4}
```

✅ Explanation: Duplicate 2 and 4 are removed.

2. Adding and Removing

```
s = {10, 20, 30}
s.add(40)      # Add new element
print(s)       # {40, 10, 20, 30}

s.remove(20)   # Remove an element
print(s)       # {40, 10, 30}
```

✓ Explanation: Sets allow adding/removing items, but they stay **unordered**.

3. Set Operations

(a) Union (combine all unique elements)

```
A = {1, 2, 3}
B = {3, 2, 5}
print(A | B)
```

Output:

```
{1, 2, 3, 5}
```

✓ Explanation: Union merges both sets, removes duplicates.

(b) Intersection (common elements)

```
print(A & B)
```

Output:

```
{3, 2}
```

✓ Explanation: Only 3 is present in both sets.

(c) Difference (in one set but not in the other)

```
print(A - B)  # {1}
print(B - A)  # {5}
```

✓ Explanation: $A - B$ = items in A but not in B.

(d) Symmetric Difference (in A or B but not both)

```
print(A ^ B)
```

Output:

```
{1, 5}
```

✓ Explanation: Elements in either set, but not in both.

Day 8

Dictionaries (Key-Value Pairs)

1. What is a dictionary?

Think of a dictionary like a **real-life dictionary**: it has a **word (key)** and a **meaning (value)**.
In Python:

```
# A simple dictionary
student = {
    "name": "Alice",    # key: name, value: Alice
    "age": 20,          # key: age, value: 20
    "city": "Delhi"     # key: city, value: Delhi
}

print(student)
```

Output:

```
{'name': 'Alice', 'age': 20, 'city': 'Delhi'}
```

2. Accessing values

You can get a value using the **key**:

```
print(student["reg"])    # Alice
print(student.get("age")) # 20
```

Tip: `.get()` is safer because it won't give an error if the key doesn't exist:

```
print(student.get("phone", "Not Found")) # Not Found
```

3. Adding or updating values

```
# Add a new key-value
student["grade"] = "A"
N v
# Update existing key
student["age"] = 21

print(student)
```

Output:

```
{'name': 'Alice', 'age': 21, 'city': 'Delhi', 'grade': 'A'}
```

4. Deleting values

```
del student["city"]      # delete by key
student.pop("grade")     # remove and return value

print(student)
```

Output:

```
{'name': 'Alice', 'age': 21}
```

5. Looping through dictionary

```
for k in student.keys():  
    print("Key:", k)  
  
for value in student.values():  
    print("Value:", value)  
  
for key, value in student.items():  
    print(key, "=", value)
```

Nested Data Structures

Sometimes, data is more **complex**. You may need a **list of dictionaries** or a **dictionary of lists**.

1. List of dictionaries

Imagine a classroom with multiple students:

```
students = [  
    {"name": "Alice", "age": 20},  
    {"name": "Bob", "age": 22}  
]  
  
    student in students:  
    print(student["name"], "-", student["age"])
```

Output:

```
Alice - 20  
Bob - 22
```

2. Dictionary of lists

A subject with multiple student marks:

```
grades = {  
    "Math": [85, 90, 78],  
    "Science": [92, 88, 79]  
}  
  
for subject, marks in grades.items():  
    print(subject, "marks:", marks)
```

Output:

```
Math marks: [85, 90, 78]  
Science marks: [92, 88, 79]
```

3. Complex nested structure

A school storing multiple students and their marks:

```
school = {  
    "class": "10th",  
    "students": [  
        {"name": "Alice", "marks": {"Math": 85, "Science": 92}},  
        {"name": "Bob", "marks": {"Math": 78, "Science": 88}}  
    ]  
}  
  
# Access Alice's Math marks  
print(school["students"][0]["marks"]["Math"]) # 85  
  
# Print all students' names and Math marks  
for student in school["students"]:  
    print(student["name"], "Math marks:", student["marks"]["Math"])
```

Output:

```
Alice Math marks: 85
Bob Math marks: 78
```

Day 9

◆ What is a Comprehension?

A **comprehension** is a short and readable way to create **lists, sets, or dictionaries** in Python — often in just one line.

Instead of using a **for** loop with `.append()`, you can write a compact expression.

1. List Comprehension

👉 Syntax:

```
[expression for item in iterable if condition]
```

Example 1 – Without comprehension:

```
numbers = [1, 2, 3, 4, 5, 6]
squares = []
for n in numbers:
    squares.append(n * n)
print(squares)
```

Example 2 – With comprehension:

```
numbers = [1, 2, 3, 4, 5, 6]
squares = [n * n for n in numbers]
print(squares)
```

Example 3 – With condition:

```
even_squares = [n * n for n in numbers if n % 2 == 0]
```

```
print(even_squares) # [4, 16, 36]
```

2. Set Comprehension

👉 Syntax is the same, but uses { }.

Example:

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_squares = {n * n for n in numbers}
print(unique_squares) # {1, 4, 9, 16, 25}
```

- ◆ Notice it removes duplicates automatically because it's a **set**.
-

3. Dictionary Comprehension

👉 Syntax:

```
{key_expression: value_expression for item in iterable if condition}
```

Example 1 – Mapping numbers to squares:

```
numbers = [1, 2, 3, 4, 5]
square_dict = {n: n * n for n in numbers}
print(square_dict) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Example 2 – With condition:

```
even_square_dict = {n: n * n for n in numbers if n % 2 == 0}
print(even_square_dict) # {2: 4, 4: 16}
```

Type	Syntax Example	Result Type
List	<code>[x*x for x in nums if x%2==0]</code>	List

Set	<code>{x*x for x in nums if x%2==0}</code>	Set
Dictionary	<code>{x: x*x for x in nums if x%2==0}</code>	Dict

🔥 So, comprehensions = **loop + if condition + expression in one line**.
They make code **shorter, cleaner, and faster**.

Day 10

1) What is a function?

👉 A function is a reusable block of code that performs a specific task. Instead of repeating the same steps, you put them in a function and call it whenever needed.

Example (Real world):

- Think of a **coffee machine**. You press the button (call function) and it always makes coffee the same way.
- In Python:

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Ava"))    # Hello, Ava!
print(greet("John"))   # Hello, John!
```

2) Parameters – ways to pass inputs

👉 Parameters are like **input slots** for a function.

a) Required (positional)

You **must** pass values in the correct order.

```
def area_rect(width, height):  
    AREA = WIDTH * HEIGHT  
    Return AREA  
  
print(area_rect(5, 4))    # 20
```

Real-world: Like filling a form → you **must** give name + age. If you skip age → error.

b) Default parameters

If you don't provide a value, Python uses the **default**.

```
def ticket_price(RATE, discount=50): # default discount 10%  
    return RATE - discount  
print(ticket_price(200))    # 150  
  
print(ticket_price(200, 70)) # 130.0
```

Real-world: An online ticket may give a **default discount** unless you choose another option.

c) Keyword arguments

Order doesn't matter if you name them.

```
def order(food, drink):  
    return f"You ordered {food} with {drink}"  
  
print(order(drink="Coke", food="Burger"))
```

Real-world: On Swiggy/Zomato, you can select "food=Pizza" and "drink=Coke" in any order.

d) Variable arguments (*args, **kwargs)

- *args = collect extra values in a tuple.
- **kwargs = collect extra named values in a dictionary.

```
def add_all(*nums):  
    return sum(nums)
```

```
print(add_all(10, 20, 30, 40))    # 100
```

**Real-world (args):* A shopping cart – you don't know how many items customer will add.

```
def profile(** info):  
    return f"{info.get('name')} works as {info.get('job')}"
```

```
print(profile(name="Alice", job="Engineer"))
```

***Real-world (kwargs):* Filling user profile → some people give "job", some don't, some give "hobby".

e) Keyword-only parameters

You **must** specify parameter names.

```
def resize(image, *, width, height):  
    return f"Resizing {image} to {width}x{height}"
```

```
print(resize("pic.jpg", width=800, height=600))
```

Real-world: Imagine an app where you must **label** size options (width=, height=). Prevents confusion.

f) Type hints

Hints about what type is expected.

```
def average(nums: list[float]) -> float:
    return sum(nums) / len(nums)
```

```
print(average([4.5, 3.0, 6.5]))    # 4.67
```

Real-world: A salary calculator → expects a list of numbers (not names).

Type hints help **other programmers** (and IDEs) understand.

3) Return values

👉 return sends results back.

```
def convert_to_usd(inr):
    return inr / 83    # assuming 1 USD = 83 INR
```

```
print(convert_to_usd(830))    # 10.0
```

- return x → gives value back.
- return alone → returns None.
- Multiple values → returned as a tuple.

Real-world: ATM → insert ₹830, it **returns \$10**.

4) Scope – where names are visible

👉 Scope decides **where a variable can be used**.

```
x = 10

def greet():
    x = 3
    print(x)
    Y = X + X
    PRINT(Y)
```

```
greet()
Y = X + X
PRINT(Y)
print(x)
```

Real-world:

- Your **home key** (local scope) only works for your house.
- **Bank key** (global scope) works everywhere.

Special cases:

- `global` → change global variable.
- `nonlocal` → change variable from outer function (useful in closures).

5) Docstrings (function documentation)

👉 A docstring is a note inside your function that explains what it does.

```
def discount(price: float, pct: float) -> float:
    """Return discounted price given original price and discount %."""
    return price * (1 - pct/100)

print(discount.__doc__)
```

Real-world: Like instructions/manual inside a machine. Future users know what the function does without opening the code.

6) Common pitfalls

👉 **Mutable defaults problem**

```
def collect(item, bucket=[]):    # BAD
    bucket.append(item)
    return bucket

print(collect(1))    # [1]
print(collect(2))    # [1, 2] ← unwanted memory sharing
```

Fix:

```
def collect(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

Real-world: If you share the same notebook with everyone, everyone writes in the same pages.

Instead → give a new notebook (list) to each person.

7) Mini Example (End-to-End)

👉 Real-world: Online shop calculates discounted price.

```
def parse_price(text: str) -> float:
    return float(text.strip().lstrip("$"))

def apply_discount(price: float, pct: float = 10.0) -> float:
    return round(price * (1 - pct/100), 2)

def summarize(*prices: float) -> tuple[int, float, float]:
    count = len(prices)
    total = round(sum(prices), 2)
    avg = round(total / count, 2) if count else 0.0
    return count, total, avg

raw = ["$12.00", " 8.5", "15.30"]
parsed = [parse_price(p) for p in raw]          # [12.0, 8.5, 15.3]
discounted = [apply_discount(p, pct=20) for p in parsed] # apply 20% off
print(summarize(*discounted))                  # (3, 28.64, 9.55)
```

- **parse_price()** → clean prices (₹ to float).
- **apply_discount()** → apply 20% offer.
- **summarize()** → count items, total, average.

Day 11

◆ Advanced Functions in Python

Types of Functions You'll Learn

User-defined functions

Functions you write yourself.

```
def add(a, b):  
    return a + b
```

- 1.
2. **Built-in functions**
Already included in Python.
Examples: `len()`, `max()`, `print()`, `sum()`, `sorted()`

Anonymous functions (lambda)

Small one-line functions.

```
square = lambda x: x * x  
print(square(5))    # 25
```

- 3.

Recursive functions

Functions that call themselves.

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)
```

- 4.
5. **Higher-order functions** Functions that take other functions as arguments (`map`, `filter`, `reduce`).

1. Default Arguments

👉 A parameter with a **default value**.

If you don't pass a value, Python uses the default.

```
def greet(name="Guest"):  
    return f"Hello, {name}!"
```

```
print(greet())          # Hello, Guest
print(greet("Ava"))     # Hello, Ava
```

✅ Useful for making arguments **optional**.

2. ***args** → Variable Positional Arguments

👉 Lets you pass **many values** into a function.
Python collects them into a **tuple**.

```
def add_numbers(*args):
    return sum(args)

print(add_numbers(2, 4, 6))      # 12
print(add_numbers(1, 2, 3, 4, 5)) # 15
```

✅ Use when you don't know how many numbers will come.

3. ****kwargs** → Variable Keyword Arguments

👉 Lets you pass **many named values** into a function.
Python collects them into a **dictionary**.

```
def show_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} → {value}")

show_info(name="Ava", age=25, city="London")
```

Output:

```
name → Ava
age → 25
city → London
```

✓ Useful for flexible **named inputs**.

4. Recursion

👉 A function that **calls itself**.

Needs a **base case** to stop, or else it goes infinite.

Example: factorial of a number

```
def factorial(n):  
    if n == 1:    # base case  
        return 1  
    return n * factorial(n - 1)  # recursive step  
  
print(factorial(5))  # 120
```

✓ Good for problems like factorials, Fibonacci, tree traversals.

5. Type Hints (New)

👉 Type hints are a way to **tell Python what types you expect** for parameters and return values.

They don't affect how Python runs, but they make code **easier to understand** and help tools (IDEs, linters).

Syntax:

```
def function_name(parameter: type) -> return_type:  
    ...
```

Example 1: Simple string type hint

```
def greet(name: str) -> str:  
    return f"Hello, {name}!"
```

```
print(greet("Ava"))    # Hello, Ava
```

- `name: str` → parameter `name` should be a string.
 - `-> str` → function should return a string.
-

Example 2: Numbers with type hints

```
def add(x: int, y: int) -> int:  
    return x + y
```

```
print(add(3, 4))    # 7
```

Example 3: List type hint

```
from typing import List
```

```
def total(scores: List[int]) -> int:  
    return sum(scores)
```

```
print(total([10, 20, 30]))    # 60
```

Day 12

Python Built-in Functions

Input / Output

- `print()` → Display output
- `input()` → Take user input

Type Conversion

- `int()`
- `float()`
- `str()`
- `bool()`
- `complex()`
- `bytes()`
- `bytearray()`
- `memoryview()`
- `list()`
- `tuple()`
- `set()`
- `frozenset()`
- `dict()`

Utility / Info

- `type()` → Get type of an object
- `id()` → Memory address
- `len()` → Length of sequence/collection
- `abs()` → Absolute value
- `round()` → Round number

- `divmod()` → Quotient & remainder in one
- `pow()` → Power (same as `**`)
- `sum()` → Sum of iterable
- `min()`, `max()` → Minimum, maximum
- `sorted()` → Sort sequence
- `reversed()` → Reverse iterator
- `enumerate()` → Index + value while looping
- `zip()` → Combine multiple iterables

Logic / Checks

- `all()` → True if all elements True
- `any()` → True if any element True
- `isinstance()` → Check object type
- `issubclass()` → Check subclass
- `callable()` → Is it callable (like function)?
- `hash()` → Hash value

Object Creation & Inspection

- `help()` → Documentation
- `dir()` → Attributes/methods of object
- `eval()` → Execute expression string
- `exec()` → Execute code string

- `globals()` → Global variables dictionary
- `locals()` → Local variables dictionary
- `vars()` → `__dict__` of object

File & System

- `open()` → Open file
- `exit()` / `quit()` → Exit interpreter

Iterators & Functional Programming

- `filter()` → Filter items with a condition
- `map()` → Apply function to items
- `next()` → Next item from iterator
- `iter()` → Create iterator
- `range()` → Sequence of numbers
- `slice()` → Slice object
- `super()` → Call parent method (OOP)

Class/Object Related

- `object()` → Base object
- `classmethod()`
- `staticmethod()`
- `property()`

Exception / Debugging

- `abs()`
- `ascii()` → Printable representation
- `bin()` → Binary string
- `oct()` → Octal string
- `hex()` → Hexadecimal string
- `format()` → Format value
- `repr()` → String representation
- `complex()` → Complex number
- `delattr()` → Delete attribute
- `getattr()` → Get attribute
- `hasattr()` → Check attribute
- `setattr()` → Set attribute
- `compile()` → Compile source code
- `chr()` → Unicode char from code
- `ord()` → Unicode code of char

◆ 1. `sorted()`

👉 Used to sort lists, tuples, or any iterable.

```
numbers = [5, 2, 9, 1]
```

```
print(sorted(numbers))          # [1, 2, 5, 9] (ascending by default)
print(sorted(numbers, reverse=True)) # [9, 5, 2, 1]
```

You can also sort strings:

```
print(sorted("hello")) # ['e', 'h', 'l', 'l', 'o']
```

◆ 2. enumerate()

👉 Adds **index numbers** to items in a loop.

```
fruits = ["apple", "banana", "cherry"]
```

```
for i, fruit in enumerate(fruits):
    print(i, fruit)
```

Output:

0 apple

1 banana

2 cherry

◆ 3. zip()

👉 Combines two (or more) lists element by element.

```
names = ["Alice", "Bob", "Charlie"]
```

```
scores = [85, 90, 78]
```

```
for name, score in zip(names, scores):
    print(name, score)
```

Output:

```
# Alice 85
# Bob 90
# Charlie 78
```

◆ 4. `any()`

👉 Returns **True** if *at least one* element is True.

```
marks = [0, 0, 70, 0]
print(any(marks))    # True (because 70 is non-zero = True)
```

◆ 5. `all()`

👉 Returns **True** only if *all* elements are True.

```
marks = [70, 80, 90]
print(all(marks))    # True (all are non-zero)
```

```
marks = [70, 0, 90]
print(all(marks))    # False (0 is False)
```

◆ 6. `doctest` – Inline Testing

👉 Lets you **test your function inside its docstring**.

```
def add(a, b):
    """
    Returns the sum of two numbers.

    >>> add(3, 3)
    6
```

```
>>> add(-1, 1)
1
"""
return a + b
```

```
if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

✓ When you run this file, `doctest` will automatically check the examples in the docstring.

- If the output matches → ✓ no error
- If it doesn't match → ✗ it throws a test failure

◆ In short

- `sorted()` → sorting
- `enumerate()` → add index in loops
- `zip()` → combine lists
- `any()` → True if at least one True
- `all()` → True if all True
- `doctest` → inline test inside docstring

◆ 1. What is a Module?

- A **module** is just a file that contains Python code (functions, variables, classes).

- Python has many built-in modules (already available), and you can also install new ones.

👉 Think of modules like **toolboxes**.

Instead of building a hammer yourself, you just open the toolbox and use the hammer.

♦ 2. Common Standard Library Modules

✓ **math** → Math toolbox

```
import math

print(math.sqrt(16))    # 4.0 (square root)
print(math.pi)         # 3.141592653589793 (value of  $\pi$ )
```

✓ **random** → Random number toolbox

```
import random

print(random.randint(1, 10)) # Random number between 1 and 10
print(random.choice(["apple", "banana", "cherry"])) # Pick a random
fruit
```

✓ **datetime** → Date & time toolbox

```
import datetime

today = datetime.date.today()
print(today)    # e.g. 2025-08-19

now = datetime.datetime.now()
print(now)      # e.g. 2025-08-19 11:45:12
```

◆ 3. Importing Modules

There are 3 main ways:

```
import math
print(math.sqrt(25))    # 5.0

from math import sqrt
print(sqrt(25))         # 5.0 (no need to write math.)

import math as m
print(m.sqrt(25))       # 5.0 (short name)
```

◆ 4. What is **pip**?

- **pip** = Python Installer for Packages.
- It helps you install modules made by other people (not built-in).
- Example: A module that can make **beautiful charts**, or connect to **databases**.

👉 Command (run in terminal, not Python shell):

```
pip install requests
```

◆ 5. Example with Third-Party Module (**requests**)

After installing:

```
import requests
```

```
response = requests.get("https://api.github.com")
print(response.status_code)    # 200 means success
```

Day 13

◆ 1. File I/O Basics

- **File** = storage of data (like a notebook).
- **I/O** = Input (read) + Output (write).
- Python needs 3 steps for File I/O:
 1. **Open** the file.
 2. **Do something** (read/write).
 3. **Close** the file.

👉 With `with open(...)` Python **auto-closes** the file (best practice).

◆ 2. Working with Text Files

A text file is just plain `.txt`.

👉 Writing into a text file

```
# Step 1: Open file in "w" mode (write mode)
with open("notes.txt", "w") as f:
    # Step 2: Write some lines
    f.write("Hello, this is my first line!\n")
    f.write("This is the second line.\n")
```

✅ Explanation:

- "w" = write mode (creates new file or erases old one).
- `f.write("...")` → writes string into file.
- `\n` = new line.

👉 After running, a file `notes.txt` will contain:

```
Hello, this is my first line!  
This is the second line.
```

Reading from a text file

```
with open("notes.txt", "r") as f:    # "r" = read mode  
    content = f.read()  
    print(content)
```

✅ Explanation:

- "r" = read mode.
- `f.read()` → reads entire file as a single string.

👉 Output:

```
Hello, this is my first line!  
This is the second line.
```

+ Appending to a file

If we don't want to erase old content, use "a" (append mode):

```
with open("notes.txt", "a") as f:  
    f.write("This line was added later!\n")
```

✅ Now file looks like:

```
Hello, this is my first line!
This is the second line.
This line was added later!
```

◆ 3. Working with CSV Files

CSV = **Comma-Separated Values** (like Excel but plain text).
Each row = one record, each column = one value.

🔧 Writing into a CSV

```
import csv

# Data to save
data = [
    ["Name", "Age", "City"],    # Header row
    ["Alice", 25, "London"],
    ["Bob", 30, "New York"],
    ["Charlie", 28, "Paris"]
]

# Open CSV in write mode
with open("people.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)    # write all rows
```

✅ Explanation:

- `csv.writer(f)` → object that writes rows.
- `writerows(data)` → writes multiple rows (list of lists).
- `newline=""` → avoids blank lines in Windows.

👉 File `people.csv` will look like:

Name, Age, City
Alice, 25, London
Bob, 30, New York
Charlie, 28, Paris

Reading from a CSV

```
import csv

with open("people.csv", "r") as basheer:
    reader = csv.reader(basheer)
    for row in reader:
        print(row)
```

Explanation:

- `csv.reader(f)` → reads CSV line by line.
- Each `row` is a Python list.

Output:

```
['Name', 'Age', 'City']
['Alice', '25', 'London']
['Bob', '30', 'New York']
['Charlie', '28', 'Paris']
```

Summary

- `"w"` = write (erases old content).
- `"r"` = read.
- `"a"` = append (adds without erasing).

- `f.read()` → read all.
- `csv.writer()` / `csv.reader()` → work with CSV files.

◆ What is **venv** in Python?

- **venv** stands for **Virtual Environment**.
- It is a tool that creates an **isolated Python environment** for a project.
- Each environment has its **own Python interpreter** and **its own libraries**.

👉 This means:

- You can install project-specific libraries **without affecting** other projects.
- Different projects can use **different versions** of the same library.

◆ Why Do We Need **venv**?

- ✓ **Avoid conflicts** → One project may need Flask 2.0, another needs Flask 3.0.
- ✓ **Keep system Python clean** → No need to install all libraries globally.
- ✓ **Reproducibility** → Team members can create the same environment using `requirements.txt`.
- ✓ **Portability** → Easy to share and run projects on other systems.

◆ How to Use **venv** (Windows)

1. Create a Virtual Environment

```
python -m venv myenv
```

👉 Creates a folder `myenv` / containing Python executable and library folders.

2. Activate the Environment

- **Command Prompt:**

```
myenv\Scripts\activate
```

- **PowerShell:**

```
.\myenv\Scripts\Activate.ps1
```

👉 You'll see `(myenv)` at the start of the terminal → means it's active.

3. Install Packages Inside venv

```
pip install flask
```

```
pip install requests
```

👉 These packages are installed **only in `myenv`**, not globally.

4. Deactivate the Environment

```
deactivate
```

👉 Returns you to the global Python.

5. Delete the Environment (if needed)

Just delete the `myenv/` folder.

◆ Real-World Example

Imagine you are working on **two projects**:

- **Project A**: Flask-based website → needs `Flask==2.0`
- **Project B**: Django-based app → needs `Django==4.0`

If you install everything globally → conflict.

With `venv`:

```
# For Project A
```

```
python -m venv flask_env
```

```
flask_env\Scripts\activate
```

```
pip install flask==2.0
```

```
# For Project B
```

```
python -m venv django_env
```

```
django_env\Scripts\activate
```

```
pip install django==4.0
```

✓ Both projects run smoothly with their own libraries.

◆ Analogy

Think of `venv` like **separate lockers in a gym**:

- Locker 1 → Shoes + Towel for Hana
- Locker 2 → Shoes + Towel for Akash

If everyone put their things in **one locker** → confusion 😓
But separate lockers keep things **organized and conflict-free**.

That's what `venv` does for Python projects 🚀

◆ Advanced: Sharing Environment (`requirements.txt`)

1. Save installed libraries:

```
pip freeze > requirements.txt
```

2. On another computer, recreate environment:

```
python -m venv newenv
```

```
newenv\Scripts\activate
```

```
pip install -r requirements.txt
```

👉 Now both systems have the **same setup**.

✅ So in short:

venv = A project's private Python workspace that keeps libraries organized, avoids conflicts, and makes projects portable.

Purpose of OOP (Object-Oriented Programming)

♦ Why We Need OOP?

Without OOP, code becomes messy when programs grow bigger.

- **Data and functions are written separately.**
- **Hard to manage when there are many entities (like 100 students, 100 cars).**

- Repeated code, less reusable, difficult to maintain.

👉 OOP solves this by organizing code into classes and objects.

♦ Purpose of OOP

1. Organization

- Groups data (attributes) and behavior (methods) together.
- Example: A **Student** object keeps both *name*, *age* (data) and *introduce()* (behavior).

2. Reusability

- **Write once, use many times by creating multiple objects.**
- **Example: One **Car** class → create 100 car objects.**

3. Scalability

- **Easier to extend and maintain when the project grows.**
- **Add new features without rewriting everything.**

4. Abstraction

- **Hides unnecessary details, shows only important features.**

- **Example: You drive a car 🚗 without knowing the engine code.**

5. Encapsulation

- **Bundles data + methods together, protects from unwanted access.**

6. Inheritance

- **Allows new classes to reuse existing class features.**
- **Example: Dog and Cat inherit from Animal.**

7. Polymorphism

- **Same method name, different behaviors depending on the object.**

- **Example: `draw()` → draws circle for Circle, rectangle for Rectangle.**

 Real-Life Analogy

- **Think of a Bank System.**

- **Customers (objects) each have attributes → name, balance.**
- **Customers also have methods → deposit, withdraw.**
- **Bank (class) manages all of them in a structured way.**

Without OOP → you'd have random variables and functions scattered everywhere.

With OOP → neat, clean, and scalable system.

 In One Line

Purpose of OOP = To make code more organized, reusable, and easier to manage for real-world problems.

Classes & Objects (`__init__`, Attributes)

◆ Concept

- **Class** → A blueprint or design (like a recipe).
- **Object** → A real-world instance created from the class (like the actual cake from the recipe).
- `__init__` → A special method that runs **automatically** when an object is created. It **initializes (sets up)** the object's data.

- **Attributes** → Variables that store the data of an object. Defined with `self`.
-

Example: Student Class

```
class Student:

    # constructor: initializes the object

    def __init__(self, name, age, grade):

        self.name = name        # attribute 1

        self.age = age          # attribute 2

        self.grade = grade      # attribute 3


    # method: behavior of the object

    def introduce(self):

        print(f"Hi, I am {self.name}, {self.age} years old, in {self.grade}.")


# creating objects

s1 = Student("Hana", 21, "Final Year")

s2 = Student("Akash", 22, "3rd Year")


# accessing attributes

print(s1.name)    # Hana
```

```
print(s2.age)      # 22
```

```
# calling method
```

```
s1.introduce()     # Hi, I am Hana, 21 years old, in Final Year.
```

```
s2.introduce()     # Hi, I am Akash, 22 years old, in 3rd Year.
```

Explanation

- `class Student`: → defines a **blueprint** for creating student objects.
- `__init__(self, name, age, grade)` → sets up each student with their own name, age, and grade.
- `self.name, self.age, self.grade` → **attributes**, unique to each student object.
- `introduce()` → **method**, shows what the object can do.
- `s1` and `s2` → two **objects**, each with different values but same structure.

Key Notes

- A **class** defines what attributes and methods the objects will have.
 - An **object** is an actual example created from the class.
 - `__init__` makes sure every object is created with proper data.
 - **Attributes** are the unique properties of each object.
 - `self` refers to the current object (it connects data to the right object).
-

👉 So in one line:

Class = design, **Object** = real instance, `__init__` = setup, **Attributes** = object's data.

Methods & Variables

♦ Instance Variables

- Belong to one object only.
- Defined inside `__init__` using `self`.
- Each object has its own copy.

👉 Example:

```
class Student:

    def __init__(self, name, age):

        self.name = name        # instance variable

        self.age = age          # instance variable


s1 = Student("Hana", 21)

s2 = Student("Akash", 22)


print(s1.name)    # Hana

print(s2.name)    # Akash
```

📌 `s1` and `s2` have different values for `name` and `age`.

♦ Class Variables

- Shared by all objects of the class.
- Defined outside `__init__` but inside the class.
- Changing it affects all objects.

👉 Example:

```
class Student:

    school = "ABC School"    # class variable (common for all)

    def __init__(self, name):

        self.name = name     # instance variable (unique per student)

s1 = Student("Hana")

s2 = Student("Akash")

print(s1.school)    # ABC School

print(s2.school)    # ABC School
```

📌 Both students belong to the same `school`.

♦ Using `self`

- `self` = refers to the current object.
- Needed to access instance variables inside methods.

- Without **self**, Python won't know which object's data to use.

👉 Example with Method:

```
class Student:

    school = "ABC School"    # class variable

    def __init__(self, name, age):

        self.name = name    # instance variable

        self.age = age

    def introduce(self):    # method

        print(f"My name is {self.name}, I study at {self.school}.")

s1 = Student("Hana", 21)

s2 = Student("Akash", 22)

s1.introduce()    # My name is Hana, I study at ABC School.

s2.introduce()    # My name is Akash, I study at ABC School.
```

📌 **self.name** → unique per student

📌 **self.school** → same for all (shared)



Teaching Notes

- **Instance Variable** → unique data per object (e.g., student's name).
 - **Class Variable** → shared data across all objects (e.g., school name).
 - **Methods** → functions inside a class that operate on object data.
 - **self** → tells Python “use this object's data.”
-

👉 In one line:

Instance variable = unique per object, Class variable = shared by all, **self** = connects data/methods to the right object.

Python OOP Concepts

♦ 9. Encapsulation

👉 What it means

- **Encapsulation** = putting data (variables) and methods (functions) inside one class.
- It also means restricting direct access to some data (by making it private).
- We only allow access through methods, not directly.

👉 Real-world example

- **Bank Account:**
 - Balance should not be changed directly.
 - Only through **deposit()** or **withdraw()**.

👉 Code Example

```
class BankAccount:

    def __init__(self, owner, balance):

        self.owner = owner

        self.__balance = balance    # private attribute

    def deposit(self, amount):

        self.__balance += amount

    def withdraw(self, amount):

        if amount <= self.__balance:

            self.__balance -= amount

        else:

            print("Insufficient funds")

    def get_balance(self):

        return self.__balance
```

Usage

```
acc = BankAccount("Hana", 1000)

acc.deposit(500)

acc.withdraw(5000)

print(acc.get_balance())    #  1500

# print(acc.__balance)      #  Not allowed (private)
```

📌 Key Idea: Encapsulation protects data. You can't just edit the balance; you must go through methods.

◆ 10. Abstraction

👉 What it means

- Abstraction = hiding unnecessary details, only showing what the user needs.
- In Python, we use abstract classes (`abc` module).

👉 Real-world example

- Car driving:
 - You just use `start()` and `drive()`.
 - You don't know how spark plugs, fuel injection, or pistons work inside.

👉 Code Example

```
from abc import ABC, abstractmethod

class Vehicle(ABC):    # Abstract class

    @abstractmethod

    def start(self):

        pass

class Car(Vehicle):

    def start(self):
```

```
        print("Car starts with a key")

class Bike(Vehicle):

    def start(self):

        print("Bike starts with a kick")

# Usage

v1 = Car()

v2 = Bike()

v1.start()    # Car starts with a key

v2.start()    # Bike starts with a kick
```

📌 **Key Idea:** Abstraction hides how “start” works inside each vehicle. You just call `.start()`.

◆ 11. Inheritance

👉 What it means

- Inheritance = a child class reuses code from a parent class.
- Saves time and avoids rewriting the same code.

👉 Real-world example

- Vehicle is the parent.
- Car and Bike are children.

- Both can `move()`, but they also have their own features.

👉 Code Example

```
class Vehicle:

    def move(self):

        print("Moving...")


class Car(Vehicle):    # inherits Vehicle

    def wheels(self):

        print("Car has 4 wheels")


class Bike(Vehicle):   # inherits Vehicle

    def wheels(self):

        print("Bike has 2 wheels")


# Usage

c = Car()

c.move()    # Moving... (from Vehicle)

c.wheels()  # Car has 4 wheels
```

📌 **Key Idea:** `Car` and `Bike` inherit the `move()` method from `Vehicle`, but they can also add their own features.

◆ 12. Polymorphism

👉 What it means

- Polymorphism = one function name → many forms.
- Different classes can define the same method name but with different behaviors.

👉 Real-world example

- Animal sounds:
 - Dog → Bark 🐕
 - Cat → Meow 🐈
 - Cow → Moo 🐮
- Same action = “make sound”, but output is different.

👉 Code Example

```
class Dog:

    def sound(self):

        print("Bark")
```

```
class Cat:

    def sound(self):

        print("Meow")
```

```
class Cow:

    def sound(self):

        print("Moo")
```

```
# Polymorphism in action
```

```
for animal in [Dog(), Cat(), Cow()]:  
    animal.sound()
```

📌 Key Idea: The method name is the same (`sound()`), but each class gives a different behavior.

◆ Exception Handling in Python — Full Breakdown

Python exceptions are runtime errors (happen when the program runs, not while writing). Instead of crashing, we use `try...except...else...finally` and `raise` to control errors.

1. `try + except` → Handling errors

This is the foundation.

```
try:  
    num = int("jhfg")    # risky  
except ValueError:  
    print("❌ Cannot convert to number")
```

👉 Output: ❌ Cannot convert to number

- **try:** contains risky code
 - **except:** runs only if that specific error happens
-

2. **else** → Runs only if no error occurs

try:

```
num = int("123")
```

except ValueError:

```
print("❌ Error converting")
```

else:

```
print("✅ No error, number is", num)
```

👉 **Output:** ✅ No error, number is 123

🔑 **Rule:** Use **else** when you want code to run only if try succeeds.

3. **finally** → Always runs (cleanup)

try:

```
file = open("data.txt", "w")
```

```
file.write("Hello")
```

except Exception as e:

```
print("Error:", e)
```

finally:

```
file.close()

print("File closed ✅")
```

Even if error happens, **finally** executes → useful for closing files, releasing DB connections, freeing resources.

4. **raise** → Manually throw an error

```
age = -3

if age < 0:

    raise ValueError("❌ Age cannot be negative")
```

👉 Output: **ValueError: ❌ Age cannot be negative**

🔑 Rule: Use **raise** when you want to enforce conditions in your program.

5. Multiple **except** Blocks → Handle different errors separately

```
try:

    x = 10 / int("0")    # risky (ZeroDivisionError)

except ZeroDivisionError:

    print("❌ Cannot divide by zero")

except ValueError:

    print("❌ Invalid conversion")
```

👉 Output: ❌ Cannot divide by zero

6. Catching Multiple Errors in One Block

```
try:
    x = int("xyz") # ValueError
except (ZeroDivisionError, ValueError) as e:
    print("❌ Error:", e)
```

👉 Output: ❌ Error: invalid literal for int() with base 10: 'xyz'

7. Catching Any Error (generic way)

```
try:
    y = 10 / 0
except Exception as e:
    print("⚠️ Something went wrong:", e)
```

👉 Output: ⚠️ Something went wrong: division by zero

⚠️ Best practice: use `Exception` when you don't know what error may happen, but log the message.

◆ Real-World Mini Example: Banking App

```
def withdraw(balance, amount):
```

```
try:
    if amount <= 0:
        raise ValueError("❌ Withdrawal amount must be positive")

    if amount > balance:
        raise ValueError("❌ Insufficient balance")

except ValueError as e:
    print("Error:", e)
else:
    balance -= amount

    print("✅ Withdrawal successful. Remaining balance:", balance)

finally:
    print("Transaction complete.\n")

return balance
```

Usage

```
bal = 500
```

```
bal = withdraw(bal, 100)    # Success
```

```
bal = withdraw(bal, 1000)   # Insufficient balance
```

```
bal = withdraw(bal, -50)    # Negative amount
```

👉 Output:

```
✅ Withdrawal successful. Remaining balance: 400
```

Transaction complete.

Error: ✖ Insufficient balance

Transaction complete.

Error: ✖ Withdrawal amount must be positive

Transaction complete.



Summary

- **try** → risky code
- **except** → handles specific errors
- **else** → runs only if no error happened
- **finally** → always runs (cleanup)
- **raise** → manually throw exceptions

Special Methods (a.k.a. Dunder Methods)

Special methods in Python start and end with double underscores (like `__str__`, `__len__`, etc.).

They give extra functionality to your custom classes and allow them to behave like built-in Python objects.

1. `__str__`

Defines what is shown when you print an object (user-friendly string).

```
class Car:

    def __init__(self, brand, model):

        self.brand = brand

        self.model = model

    def __str__(self): # User-friendly string

        return f"{self.brand} {self.model}"

car1 = Car("Bajaj", "Pulsar 125")

print(car1) # 👉 Bajaj Pulsar 125
```

Without `__str__`, printing an object shows memory location like `<__main__.Car object at 0x...>`

2. `__repr__`

Defines the “developer-friendly” representation of the object (useful for debugging).

```
class Car:

    def __init__(self, brand, model):

        self.brand = brand

        self.model = model
```

```
def __repr__(self): # Debug info
    return f"Car(brand='{self.brand}', model='{self.model}')"

car1 = Car("Bajaj", "Pulsar 125")

print(repr(car1)) # 👉 Car(brand='Bajaj', model='Pulsar 125')
```

3. `__len__`

Defines what happens when you use `len()` on an object.

```
class Playlist:
    def __init__(self, songs):
        self.songs = songs

    def __len__(self):
        return len(self.songs)

pl = Playlist(["Song1", "Song2", "Song3"])

print(len(pl)) # 👉 3
```

4. `__eq__`

Defines how two objects are compared using `==`.

```
class Bike:
```

```
def __init__(self, brand, cc):  
    self.brand = brand  
    self.cc = cc  
  
def __eq__(self, other):  
    return self.cc == other.cc # compare based on cc  
  
bike1 = Bike("Pulsar", 125)  
bike2 = Bike("Shine", 125)  
  
print(bike1 == bike2) # 👉 True (because cc is same)
```

✅ Summary:

- `__str__` → User view
- `__repr__` → Developer view
- `__len__` → Length meaning
- `__eq__` → Equality meaning

Lambda & Functional Tools: every step, explained clearly

Nice — I'll walk through **lambda**, **map**, **filter**, **reduce**, and **zip** showing *what happens at each step* so nothing is mysterious. Each section: short definition → code → **step-by-step execution** → notes & common pitfalls.

Lambda (anonymous function)

- **What it is:** a tiny one-line function with no name. Use for short operations you don't need to reuse.
- **Syntax:** `lambda args: expression`

Example

```
square = lambda a,b: a + b
```

```
print(square(5,7))    # 25
```

Step-by-step:

1. `lambda x: x * x` — defines an anonymous function that takes one argument `x` and returns `x * x`.
2. `square = ...` — binds that function object to the name `square`.
3. `square(5)` — calls the function with `x = 5`.
4. The expression `x * x` is evaluated → `5 * 5 = 25`.
5. `25` is returned and printed.

Notes:

- Lambdas can have multiple args: `lambda x, y: x + y`.
- They can only contain an expression (no statements like `if` block, `return` etc.). For complex logic, use `def`.

`map(function, iterable, ...)`

- **What it does:** applies `function` to each element of the iterable(s) and returns a lazy iterator (in Python 3).
- **Common use:** transform each item in a list.

```
numbers = [1, 2, 3, 4]

result = map(lambda x: x * 2, numbers)

print(list(result))    # [2, 4, 6, 8]
```

Step-by-step:

1. `map(...)` creates a map-object (an iterator) but does **not** execute yet.
2. `list(result)` requests items from the iterator.
3. For the first item:
 - take `1` from `numbers`
 - call `lambda x: x*2` with `x=1`
 - get `2`, yield it
4. Repeat for `2` → yield `4`; for `3` → `6`; for `4` → `8`.
5. `list()` collects yielded values into `[2, 4, 6, 8]` and prints them.

Multi-iterable use:

```
a = [1, 2]

b = [10, 20]

list(map(lambda x, y: x+y, a, b))    # [11, 22]
```

- Step-by-step: pairs (1,10) → 11, (2,20) → 22. Stops at shortest iterable.

Pitfalls / notes:

- `map` returns an iterator; if you print it directly you get a map-object. Convert to `list()` to inspect.
 - Equivalent to list comprehension: `[f(x) for x in iterable]` — often clearer.
-

`filter(function, iterable)`

- **What it does:** keeps only items where `function(item)` is truthy; returns a lazy iterator.
- **Common use:** select items that satisfy a condition.

```
numbers = [10, 15, 20, 25, 30]
evens = filter(lambda x: x % 2 == 0, numbers)
print(list(evens))    # [10, 20, 30]
```

Step-by-step:

1. `filter(...)` creates a filter-object (iterator).
2. `list(evens)` starts consuming:
 - take 10 → call predicate `10 % 2 == 0` → True → 10 is yielded.
 - take 15 → predicate False → skip (not yielded).
 - continue for 20 → yield 20; etc.
3. Result list `[10, 20, 30]`.

Shortcut: `filter(None, iterable)` removes falsy values (0, '', None, False).

Equivalent: `[x for x in iterable if predicate(x)]`.

`reduce(function, iterable[, initial])` — from `functools`

- **What it does:** collapses an iterable to a single value by applying `function` cumulatively.
- Must import: `from functools import reduce`.

```
from functools import reduce

numbers = [1, 2, 3, 4]

sum_result = reduce(lambda acc, x: acc + x, numbers)

print(sum_result)    # 10
```

Step-by-step (no initial):

1. Start with first two elements: `acc = 1, x = 2`.
2. Compute `acc + x → 1 + 2 = 3`. New `acc = 3`.
3. Next element `x = 3`: `acc + x → 3 + 3 = 6`. New `acc = 6`.
4. Next element `x = 4`: `6 + 4 = 10`. New `acc = 10`.
5. Iterable exhausted → `10` is returned.

Step-by-step (with `initial`):

```
reduce(lambda acc, x: acc + x, [1,2,3], 10)    # starts acc=10

# steps: 10+1=11, 11+2=13, 13+3=16 => returns 16
```

- Here `acc` starts as 10, then the first element is 1.

Notes & common uses:

- Use for sums/products, flattening, combining items.
 - With no `initial`, reduction starts with the first two list items — list must be non-empty. With `initial`, can handle empty lists.
 - `reduce` is less readable for complex operations; sometimes `sum()` / list comprehensions are clearer.
-

**zip(iterables)*

- **What it does:** pairs up elements from multiple iterables element-wise into tuples. Returns an iterator in Python 3.

```
names = ["A", "B", "C"]  
scores = [90, 80, 70]  
z = zip(names, scores)  
print(list(z))    # [('A', 90), ('B', 80), ('C', 70)]
```

Step-by-step:

1. `zip` creates an iterator that on each step pulls the next item from each iterable.
2. First step: take "A" and 90 → produce ('A', 90).
3. Next: ('B', 80) then ('C', 70).
4. Stops when the shortest iterable is exhausted.

Unzipping:

```
pairs = [('A',90), ('B',80)]
```

```
names, scores = zip(*pairs)    # names=('A','B'), scores=(90,80)
```

Notes:

- To avoid truncation use `itertools.zip_longest()` which fills missing with a value.
- Good for iterating two lists in parallel: `for a, b in zip(list1, list2): ...`

Combined real-world mini example — explained step-by-step

Task: scale marks by +10, keep passing marks (≥ 50), compute total of passing marks, pair names with passing marks.

```
from functools import reduce
```

```
names = ["Alice", "Bob", "Cara", "Dan"]
```

```
marks = [35, 45, 70, 48]
```

```
# 1) scale marks
```

```
scaled_iter = map(lambda m: m + 10, marks)           # map returns  
iterator
```

```
# 2) filter passing ( $\geq 50$ )
```

```
passed_iter = filter(lambda m: m >= 50, scaled_iter) # chained  
iterator
```

```

# 3) convert to list to reuse values (we need them twice)

passed_marks = list(passed_iter)    # evaluate the chained iterators
now

# 4) total using reduce

total_passed = reduce(lambda a, b: a + b, passed_marks, 0)

# 5) pair names & passed marks: but we lost which names corresponded;
better to zip before filtering

print("passed_marks:", passed_marks)

print("total_passed:", total_passed)

```

Step-by-step explanation of the correct approach (preserve name–mark pairs):

1. `zip(names, marks)` → iterator of pairs: `('Alice', 35), ('Bob', 45), ('Cara', 70), ('Dan', 48)`.
2. Use `map` or comprehension to scale marks inside pairs:
 - `map(lambda nm: (nm[0], nm[1]+10), zipped)` produces `('Alice', 45), ('Bob', 55), ('Cara', 80), ('Dan', 58)`.
3. `filter(lambda nm: nm[1] >= 50, ...)` keeps `('Bob', 55), ('Cara', 80), ('Dan', 58)`.
4. Convert to list to reuse: `passed_list = list(...)` → `[('Bob', 55), ('Cara', 80), ('Dan', 58)]`.
5. `reduce` to compute total: reduce over mark component:
 - map to marks: `map(lambda nm: nm[1], passed_list)` → `[55, 80, 58]`

- `reduce(lambda a,b: a+b, [55,80,58], 0) → 193.`

This preserves names and shows how `zip`, `map`, `filter`, and `reduce` can be combined safely.

Quick tips & pitfalls to remember

- `map/filter/zip` return **iterators** in Python 3 — convert to `list()` to inspect.
- `reduce` is in `functools`.
- `zip` stops at shortest iterable; use `zip_longest` if you need padding.
- `map` with multiple iterables stops at shortest too.
- Prefer **list comprehensions** for readability unless you need a lazy iterator.
 - `map(f, it) ≈ [f(x) for x in it]`
 - `filter(p, it) ≈ [x for x in it if p(x)]`
- Use `lambda` for tiny, throwaway functions; use `def` for clarity if logic is more than a simple expression.

1. Multithreading

👉 **Definition:** Running multiple *threads* (lightweight processes) at the same time inside one program.

- Threads share the same memory but run different parts of code.
- Good for tasks like **waiting for files, making network requests, user interface tasks**.
- Not great for heavy CPU work in Python (because of the **GIL – Global Interpreter Lock**).

Example: Multithreading in Python

```
import threading

import time


def print_numbers():

    for i in range(1, 6):

        print("Number:", i)

        time.sleep(1)  # simulate work


def print_letters():

    for ch in ['A', 'B', 'C', 'D', 'E']:

        print("Letter:", ch)

        time.sleep(1)


# Create threads

t1 = threading.Thread(target=print_numbers)

t2 = threading.Thread(target=print_letters)


# Start threads

t1.start()

t2.start()


# Wait for them to finish
```

```
t1.join()

t2.join()


print("Done!")
```

What happens:

- Instead of printing numbers first then letters, both run **side by side**.
- Output may interleave like:

```
Number: 1
Letter: A
Number: 2
Letter: B
...
```

📌 **Real-world use case:** Downloading multiple files at the same time, while still being able to use the app.

2. Multiprocessing (maybe you meant *multiloading* → usually called multiprocessing)

👉 **Definition:** Running multiple *processes* (separate Python interpreters).

- Each process has its own memory.

- Good for **CPU-heavy tasks** like calculations, image processing, ML training.
- Avoids the GIL problem.

Example: Multiprocessing in Python

```
import multiprocessing

import time

def square_numbers():
    for i in range(1, 6):
        print(f"Square of {i}: {i*i}")
        time.sleep(1)

if __name__ == "__main__":
    # Create 2 processes
    p1 = multiprocessing.Process(target=square_numbers)
    p2 = multiprocessing.Process(target=square_numbers)

    p1.start()
    p2.start()

    p1.join()
    p2.join()
```

```
print("Done!")
```

What happens:

- Two independent processes run, both calculating squares.
- Runs in *true parallel* (utilizes multiple CPU cores).

📌 **Real-world use case:** Processing large datasets faster by splitting work across CPUs.

⚡ 3. Async / Await (Asynchronous programming)

👉 **Definition:** A modern way in Python to run **non-blocking code** (one task can pause while waiting, and another runs).

- Useful for **I/O-bound tasks** like web requests, database queries, reading files.
- You declare async functions with `async def` and pause with `await`.

Example: Async / Await in Python

```
import asyncio
```

```
async def task1():
```

```
    print("Start Task 1")
```

```
    await asyncio.sleep(2) # non-blocking wait
```

```
    print("End Task 1")
```

```
async def task2():  
    print("Start Task 2")  
    await asyncio.sleep(1)  
    print("End Task 2")  
  
async def main():  
    await asyncio.gather(task1(), task2())  
  
asyncio.run(main())
```

What happens:

- Task1 waits for 2 sec, Task2 waits for 1 sec.
- They overlap → total time $\approx$ 2 sec (not 3).
- Output order:

Start Task 1

Start Task 2

End Task 2

End Task 1

 **Real-world use case:** A web server handling thousands of requests at the same time without creating thousands of threads.

✨ Summary (like telling to a child)

- **Multithreading** → Many people sharing the same kitchen (same memory), each cooking something small (I/O tasks).
- **Multiprocessing** → Many people in different kitchens (separate memory), each cooking a big dish (CPU-heavy tasks).
- **Async/Await** → One chef who is smart: while water boils, he chops vegetables (switches tasks efficiently).

PythonSyllabus

Python Syllabus

40-Day Python Mastery Syllabus (80 Hours)

◆ Week 1: Core Basics

Goal: Establish strong foundations in syntax, logic, and problem solving.

- Day 1: Intro to Programming
 - What is programming? Why Python?
 - Installing Python, VS Code / PyCharm
 - Writing your first program: `print("Hello, world!")`
- Day 2: Variables and Data Types
 - int, float, str, bool

- `print()`, `input()`, type conversion
- Day 3: Operators
 - Arithmetic, Comparison, Logical, Assignment
 - Intro to Bitwise Operators
- Day 4: Conditionals
 - `if`, `elif`, `else`, nesting, indentation rules
- Day 5: Loops
 - `while`, `for`, `range()`
 - Loop control: `break`, `continue`, `else`
- Day 6: Practice Day
 - Solve small logic-based problems and quizzes
- Day 7: Mini Project

- CLI Calculator, Password Generator, or Number Guess Game
-

- ◆ Week 2: Data Structures Deep Dive

Goal: Master Python collections and manipulate complex data efficiently.

- Day 8: Lists
 - Creation, indexing, slicing, methods (`append()`, `extend()`, etc.)
- Day 9: Tuples
 - Immutability, tuple unpacking, when to use
- Day 10: Sets
 - Unique values, set operations: union, intersection, difference
- Day 11: Dictionaries

- Key-value pairs, CRUD, `get()`, looping
(`items()`, `keys()`, `values()`)
 - Day 12: Nested Data Structures
 - List of dicts, dict of lists, parsing complex data
 - Day 13: Comprehensions
 - List, set, dict comprehensions with conditions
 - Day 14: Project
 - Contact Book or Quiz App (focus: lists/dicts)
-

◆ Week 3: Functions, Modules, and Files

Goal: Write reusable, organized code and handle external files.

- Day 15: Functions
 - Defining, parameters, return, scope, docstrings

- Day 16: Advanced Functions

- Default args, `*args`, `**kwargs`, recursion
- ♦ New: Intro to Type Hints (`name: str -> str`)

- Day 17: Built-in Functions

- `sorted()`, `enumerate()`, `zip()`, `any()`, `all()`
- ♦ New: `doctest` – inline testing for functions

- Day 18: Modules & Standard Library

- `math`, `random`, `datetime`, importing modules
- ♦ New: Intro to `pip` & installing third-party modules

- Day 19: File I/O

- Reading/writing text files and CSVs using `csv` module
 - Day 20: Project
 - To-Do App or Student Grade Tracker
 - ♦ New: Setup `venv` and `requirements.txt`
-

♦ Week 4: OOP and Exception Handling

Goal: Learn object-oriented programming and error management.

- Day 21: Classes & Objects
 - `__init__`, object attributes
- Day 22: Methods & Variables
 - Instance vs class variables, using `self`
- Day 23: Inheritance

- `super()`, method overriding
- Day 24: Encapsulation
 - Private variables, `@property` decorator
- Day 25: Special Methods
 - `__str__`, `__repr__`, `__len__`, `__eq__`, etc.
- Day 26: Exception Handling
 - `try`, `except`, `else`, `finally`, `raise`
- Day 27: Project
 - Library Management System or Banking System
 - ♦ Optional Bonus: GUI version with `tkinter`

♦ Week 5: Intermediate/Advanced Concepts

Goal: Master functional programming, file formats, and regex.

- Day 28: Lambda & Functional Tools
 - `lambda`, `map()`, `filter()`, `reduce()`, `zip()`
- Day 29: Decorators
 - Basic decorators and decorators with arguments
- Day 30: Iterators & Generators
 - `__iter__`, `__next__`, `yield`
- Day 31: JSON
 - Parsing and saving structured data with `json` module
- Day 32: DateTime & Timedelta
 - Countdown app, timer, formatted time
- Day 33: Regex & Logging
 - Pattern matching, extraction, validation with `re`

- ◆ New: Intro to `logging` module
 - Day 34: Project
 - Expense Tracker or Log Analyzer
 - ◆ New: CLI project with `argparse` for command-line arguments
-

◆ Week 6: Algorithms, Testing, and Final Projects

Goal: Learn how to test, structure, and build real-world Python apps.

- Day 35: Algorithms
 - Sorting, searching, recursion – Pythonic implementations
- Day 36: Testing
 - Unit testing with `unittest`, test cases, mocking
 - Review `doctest` and best practices

- Day 37: Code Organization
 - `__name__ == "__main__"`, reusable functions, module structuring
- Day 38: Virtual Environments, Pip, Packaging
 - Project structure, `setup.py`, `venv`, `pip install -e .`
- Day 39: Final Project – Day 1
 - Choose one:
 - Blog Engine CLI
 - Inventory Management System
 - Flashcard Learning App
- Day 40: Final Project – Day 2
 - Wrap-up, testing, debugging, peer review or presentation

Module 1: Introduction

- What is SQL?
 - Why SQL is important for web apps
 - SQL vs. NoSQL (quick comparison)
 - Client tools: MySQL Workbench / pgAdmin / SQLite Browser
-

Module 2: DDL (Data Definition Language)

👉 Used to **create & modify database structure**

- CREATE DATABASE, DROP DATABASE
- CREATE TABLE, ALTER TABLE, DROP TABLE
- Data Types (INT, VARCHAR, DATE, BOOLEAN, JSON)
- Constraints:

- PRIMARY KEY, FOREIGN KEY
 - NOT NULL, UNIQUE, CHECK, DEFAULT
 - Real-world: Designing tables for **users, products, orders**
-

Module 3: DML (Data Manipulation Language)

👉 Used to **manipulate data in tables**

- INSERT INTO
- UPDATE
- DELETE
- SELECT (basic queries with WHERE)
- Sorting with ORDER BY
- Pagination with LIMIT, OFFSET

- Real-world: Register user, update profile, delete inactive accounts

Module 4: DQL (Data Query Language – part of DML but often separate)

👉 Focused on **retrieving data**

- Filtering: **WHERE, BETWEEN, IN, LIKE, IS NULL**
- Aggregate functions: **COUNT, SUM, AVG, MIN, MAX**
- **GROUP BY, HAVING**
- Real-world: Count active users, total sales, posts per category

Module 5: Joins & Relationships

- **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN**

- Multi-table joins
 - Self join
 - Real-world: Get all users and their orders, posts with comments
-

Module 6: Subqueries & CTE

- Subqueries in **SELECT**, **WHERE**, **FROM**
 - **EXISTS** vs. **IN**
 - Common Table Expressions (**WITH**)
 - Real-world: Find users with orders above average value
-

Module 7: DCL (Data Control Language)

 Security & permissions

- GRANT
 - REVOKE
 - User roles in MySQL/Postgres
 - Real-world: Restrict access for web app DB user
-

Module 8: TCL (Transaction Control Language)

👉 Handling **transactions & consistency**

- START TRANSACTION / BEGIN
 - COMMIT
 - ROLLBACK
 - SAVEPOINT
 - Real-world: Shopping cart → checkout (all-or-nothing)
-

Module 9: Functions & Operators

- String functions: `CONCAT`, `UPPER`, `LOWER`, `SUBSTRING`, `LIKE`
 - Date/time functions: `NOW()`, `DATEDIFF`, `EXTRACT`
 - Math functions: `ROUND`, `CEIL`, `FLOOR`, `ABS`
 - Conditional expressions: `CASE`, `COALESCE`
 - Real-world: Format product prices, show “time since last login”
-

Module 10: Views, Indexes & Performance

- `CREATE VIEW`, `DROP VIEW`
- `CREATE INDEX`, `DROP INDEX`
- `EXPLAIN` query plan

- Real-world: Dashboard views, speeding up search queries

Assessments

English

batch_19

Sql_Notes

Introduction to SQL

1. What is SQL?

♦ **SQL (Structured Query Language)** is a language used to **communicate with relational databases**.

- Think of a **database** like a big Excel sheet, but smarter:
 - It stores data in **tables (rows & columns)**.
 - It allows fast searching, filtering, and combining of data.
- SQL helps you **ask questions to the database** and **manipulate data**.

👉 Example:

Suppose you have a **users table** in your web app:

id	name	email	created_at
1	Ananth	ananth@mail.com	2025-09-20
2	Priya	priya@mail.com	2025-09-21

SQL query to get all users:

```
SELECT * FROM users;
```

Result:

```
1 | Ananth | ananth@mail.com | 2025-09-20
2 | Priya | priya@mail.com | 2025-09-21
```

2. Why SQL is Important for Web Apps

Most web apps (Instagram, Amazon, Twitter, etc.) store and retrieve data using SQL.

- ♦ Some examples in **web development**:

Login page → check if email & password exist in database.

```
SELECT * FROM users WHERE email = 'ananth@mail.com';
```

-

E-commerce site → fetch all products in a category.

```
SELECT name, price FROM products WHERE category = 'Shoes';
```

-

Analytics dashboard → total number of users.

```
SELECT COUNT(*) FROM users;
```

-

👉 Without SQL, your app cannot store users, products, or transactions reliably.

3. SQL vs. NoSQL (Quick Comparison)

Feature	SQL (Relational DBs)	NoSQL (Document/Key-Value DBs)
Structure	Tables (rows & columns)	JSON-like docs, key-value pairs
Schema	Fixed schema (strict)	Flexible (schema-less)
Best for	Banking, e-commerce, structured data	Social feeds, chat apps, unstructured data
Examples	MySQL, PostgreSQL, SQLite	MongoDB, Firebase, Cassandra
Joins	Supports complex joins	Usually no joins
Use Case	Order management, payments	User sessions, activity logs

👉 **Example:**

SQL (MySQL/Postgres):

```
SELECT * FROM orders WHERE user_id = 1;
```

-

NoSQL (MongoDB):

```
{ "user_id": 1, "orders": [ {"product": "Laptop", "price": 50000} ] }
```

-

- ♦ For **web developers**, SQL is usually the **first choice** because:

- Strong consistency
- Easy joins between related data
- Works with most frameworks (Django, Laravel, Express, Spring Boot, etc.)

4. Client Tools

To work with SQL databases, you use **client tools**. These tools help you connect to the database and run queries.

1. MySQL Workbench

- For MySQL databases
- GUI tool: lets you create tables, run queries, visualize schemas
- Example: Open `products` table → run `SELECT * FROM products;`

2. pgAdmin

- For PostgreSQL databases
- Web-based GUI tool
- Example: Write SQL queries, view execution plans

3. SQLite Browser (DB Browser for SQLite)

- For SQLite databases (lightweight, file-based DB often used in mobile/web apps)
- Example: Open a `.db` file used in a Django/Flask project → explore tables

✓ Summary

- SQL = Language to talk to relational databases
- Essential for **web apps** → login, products, orders, analytics
- SQL vs NoSQL → structured vs flexible data
- Tools: MySQL Workbench, pgAdmin, SQLite Browser

DDL (Data Definition Language)

👉 DDL is used to **define and change the structure** of a database.

It doesn't deal with actual data (like inserting or updating rows), but with the **blueprint** of the database.

1. Create & Drop Database

- **CREATE DATABASE** → Makes a new database.
- **DROP DATABASE** → Deletes an existing database (and all tables inside).

Example:

- `-- Create a database`
- `CREATE DATABASE ecommerce;`
-

- `-- Use the database`
 - `USE ecommerce;`
 -
 - `-- Delete the database`
 - `DROP DATABASE ecommerce;`
-

2. Create, Alter, Drop Table

- **CREATE TABLE** → Creates a new table.
- **ALTER TABLE** → Modifies an existing table (add/modify/drop columns).
- **DROP TABLE** → Deletes a table completely.

Example:

- `-- Create a users table`
- `CREATE TABLE users (`
- `user_id INT PRIMARY KEY,`
- `name VARCHAR(100),`
- `email VARCHAR(100) UNIQUE,`
- `created_at DATE`
- `);`
-
- `-- Add a new column`
- `ALTER TABLE users ADD phone VARCHAR(15);`
-
- `-- Modify column type`
- `ALTER TABLE users MODIFY phone VARCHAR(20);`
-
- `-- Delete the column`
- `ALTER TABLE users DROP COLUMN phone;`
-
- `-- Delete the table`
- `DROP TABLE users;`

3. Data Types

Data types define what kind of values a column can hold.

- **INT** → Whole numbers (e.g., 1, 50, -20)
- **VARCHAR(n)** → Variable-length string (e.g., "John")
- **DATE** → Calendar date (YYYY-MM-DD)
- **BOOLEAN** → True/False values
- **JSON** → Stores structured data in JSON format

Example:

- `CREATE TABLE products (`
- `product_id INT PRIMARY KEY,`
- `name VARCHAR(100) NOT NULL,`
- `price DECIMAL(10,2),`
- `available BOOLEAN,`
- `details JSON`
- `);`

4. Constraints

Constraints enforce **rules** on data.

- **PRIMARY KEY** → Unique ID for each row (cannot be NULL).
- **FOREIGN KEY** → Link between two tables.
- **NOT NULL** → Value must always be given.

- **UNIQUE** → No duplicate values allowed.
- **CHECK** → Ensure condition is true.
- **DEFAULT** → Provide a default value if not given.

Example:

- `-- Orders table linked with Users`
 - `CREATE TABLE orders (`
 - `order_id INT PRIMARY KEY,`
 - `user_id INT,`
 - `order_date DATE DEFAULT CURRENT_DATE,`
 - `amount DECIMAL(10,2) CHECK (amount > 0),`
 -
 - `-- Foreign Key: user_id must exist in users table`
 - `FOREIGN KEY (user_id) REFERENCES users(user_id)`
 - `);`
-

5. Real-world Example: E-commerce Database

You're designing a database for an online shop.

- **Users Table** → store customer info.
- **Products Table** → store product details.
- **Orders Table** → link users with their purchases.
- `CREATE TABLE users (`
- `user_id INT PRIMARY KEY,`
- `name VARCHAR(100) NOT NULL,`
- `email VARCHAR(100) UNIQUE,`
- `created_at DATE DEFAULT CURRENT_DATE`
- `);`
-
- `CREATE TABLE products (`

- product_id INT PRIMARY KEY,
- name VARCHAR(100) NOT NULL,
- price DECIMAL(10,2) CHECK (price > 0),
- stock INT DEFAULT 0
-);
-
- CREATE TABLE orders (
- order_id INT PRIMARY KEY,
- user_id INT NOT NULL,
- product_id INT NOT NULL,
- quantity INT CHECK (quantity > 0),
- order_date DATE DEFAULT CURRENT_DATE,
-
- FOREIGN KEY (user_id) REFERENCES users(user_id),
- FOREIGN KEY (product_id) REFERENCES products(product_id)
-);

DML (Data Manipulation Language)

👉 DML is used to insert, update, delete, and query data in tables.

1. INSERT INTO

Used to **add new records** into a table.

Example:

```
-- Insert a new user

INSERT INTO users (user_id, name, email)

VALUES (1, 'John Doe', 'john@example.com');
```

```
-- Insert multiple users at once

INSERT INTO users (user_id, name, email)

VALUES

(2, 'Alice Smith', 'alice@example.com'),

(3, 'Bob Lee', 'bob@example.com');
```

2. UPDATE

Used to **change existing data**.

Example:

```
-- Update a user's email

UPDATE users

SET email = 'john.doe@example.com'

WHERE user_id = 1;


-- Update multiple fields

UPDATE users

SET name = 'Alice Johnson', email = 'alice.j@example.com'

WHERE user_id = 2;
```

 Always use **WHERE** when updating, otherwise **all rows** will be updated!

3. DELETE

Used to **remove records**.

Example:

```
-- Delete one user
```

```
DELETE FROM users
```

```
WHERE user_id = 3;
```

```
-- Delete all users (⚠ dangerous!)
```

```
DELETE FROM users;
```

4. SELECT (Basic Queries with WHERE)

Used to **retrieve data**.

Example:

```
-- Get all users
```

```
SELECT * FROM users;
```

```
-- Get specific columns
```

```
SELECT name, email FROM users;
```

```
-- Get a single user
```

```
SELECT * FROM users WHERE user_id = 1;
```

```
-- Filter with condition
```

```
SELECT * FROM users WHERE name = 'Alice Johnson';
```

5. Sorting with ORDER BY

Used to **arrange results**.

Example:

```
-- Sort users by name (ascending)
```

```
SELECT * FROM users ORDER BY name ASC;
```

```
-- Sort users by created date (newest first)
```

```
SELECT * FROM users ORDER BY created_at DESC;
```

6. Pagination with LIMIT, OFFSET

Used when showing results in **pages** (common in web apps).

Example:

```
-- Get first 5 users
```

```
SELECT * FROM users LIMIT 5;
```

```
-- Get next 5 users (page 2)

SELECT * FROM users LIMIT 10 OFFSET 5;
```

7. Real-world Scenarios

Register user (INSERT)

```
INSERT INTO users (user_id, name, email)

VALUES (4, 'Charlie Brown', 'charlie@example.com');
```

Update profile (UPDATE)

```
UPDATE users

SET name = 'Charlie B.'

WHERE user_id = 4;
```

Delete inactive accounts (DELETE)

```
DELETE FROM users

WHERE created_at < '2024-01-01';
```

Show users (with pagination) (SELECT + LIMIT/OFFSET)

```
SELECT user_id, name, email

FROM users

ORDER BY created_at DESC

LIMIT 10 OFFSET 20;    -- Page 3 (skip 20, get next 10)
```

✓ Summary

- `INSERT INTO` → Add new rows.
- `UPDATE` → Modify existing rows.
- `DELETE` → Remove rows.
- `SELECT` → Query data (filter with `WHERE`).
- `ORDER BY` → Sort results.
- `LIMIT, OFFSET` → Paginate results.

DQL (Data Query Language)

👉 DQL is mainly about **retrieving data** from the database using `SELECT` queries.

While DML includes `INSERT`, `UPDATE`, `DELETE`, DQL focuses on **fetching** and analyzing data.

1. Filtering with `WHERE`

`WHERE` = Apply conditions to rows.

```
-- Find users who are active
SELECT user_id, name
FROM users
WHERE status = 'active';
```

📌 **Explanation:** Only rows with `status = 'active'` are returned.

2. BETWEEN, IN, LIKE, IS NULL

- **BETWEEN** → Range filter

```
-- Orders placed in September 2025
SELECT order_id, order_date, amount
FROM orders
WHERE order_date BETWEEN '2025-09-01' AND '2025-09-30';
```

- **IN** → Match against a list

```
-- Find users from specific cities
SELECT name, city
FROM users
WHERE city IN ('Chennai', 'Bangalore', 'Delhi');
```

- **LIKE** → Pattern matching

```
-- Find users whose name starts with 'A'
SELECT name, email
FROM users
WHERE name LIKE 'A%';
```

- **IS NULL** → Check missing values

```
-- Find users without phone number
SELECT name
FROM users
WHERE phone IS NULL;
```

3. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

- **COUNT**

```
-- Count total active users
SELECT COUNT(*) AS total_active_users
FROM users
WHERE status = 'active';
```

- **SUM**

```
-- Total sales amount
SELECT SUM(amount) AS total_sales
FROM orders;
```

- **AVG**

```
-- Average order value
SELECT AVG(amount) AS avg_order_value
FROM orders;
```

- **MIN / MAX**

```
-- Smallest & largest order amount
SELECT MIN(amount) AS min_order, MAX(amount) AS max_order
FROM orders;
```

4. **GROUP BY** – Grouping Data

```
-- Count orders per user
SELECT user_id as id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

📌 Groups rows by `user_id` and counts orders for each.

5. HAVING – Filter after grouping

```
-- Find users with more than 5 orders
SELECT user_id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id
HAVING COUNT(*) > 5;
```

📌 `HAVING` works like `WHERE`, but on grouped/aggregated data.

6. Real-world Examples

- **Active Users Count** → `COUNT(*) WHERE status='active'`
 - **Total Sales** → `SUM(amount)`
 - **Posts per Category** → `GROUP BY category`
 - **High-value Customers** → `HAVING SUM(amount) > 10000`
-

✅ Summary

- `WHERE` → filter rows
- `BETWEEN, IN, LIKE, IS NULL` → advanced filters
- `COUNT, SUM, AVG, MIN, MAX` → aggregate functions
- `GROUP BY` → group results

- **HAVING** → filter grouped results

Joins & Relationships

1 What are Joins?

- **Joins** combine rows from two or more tables based on a related column.
- Common use case: connecting **users** → **orders**, **posts** → **comments**, etc.

👉 Example schema:

```
-- Users table
id | name
---|-----
1  | Alice
2  | Bob
3  | Charlie

-- Orders table
id | user_id | product
---|-----|-----
1  | 1       | Laptop
2  | 2       | Phone
3  | 2       | Tablet
4  | 4       | Camera  -- notice: user_id=4 doesn't exist in users
```

2 INNER JOIN

- Returns rows that have **matching values in both tables**.
- If no match → row is excluded.

```
SELECT users.name, orders.product
FROM users
INNER JOIN orders
ON users.id = orders.user_id;
```

✅ Output:

```
name      | product
-----|-----
Alice     | Laptop
Bob        | Phone
Bob        | Tablet
```

👉 User **Charlie** is missing (no orders).

👉 Order with **user_id=4** is missing (no matching user).

3 LEFT JOIN

- Returns **all rows from the left table** + matching rows from the right.
- If no match, right side = **NULL**.

```
SELECT users.name, orders.product
FROM users
LEFT JOIN orders
ON users.id = orders.user_id;
```

✅ Output:

```
name      | product
-----|-----
Alice     | Laptop
Bob        | Phone
Bob        | Tablet
Charlie    | NULL
```

👉 Charlie shows up, but with **NULL product** (no orders).

4 RIGHT JOIN

- Opposite of LEFT JOIN.
- Returns **all rows from the right table** + matching from the left.

```
SELECT users.name, orders.product
FROM users
RIGHT JOIN orders
ON users.id = orders.user_id;
```

✅ Output:

```
name      | product
-----|-----
Alice     | Laptop
Bob       | Phone
Bob       | Tablet
NULL      | Camera
```

👉 The **Camera order** appears, but user = **NULL** (no matching user).

5 FULL JOIN

- Returns **all rows when there's a match in either table**.
- If no match, fills with **NULL**.

```
SELECT users.name, orders.product
FROM users
FULL JOIN orders
ON users.id = orders.user_id;
```

✓ Output:

name	product
Alice	Laptop
Bob	Phone
Bob	Tablet
Charlie	NULL
NULL	Camera

6 Multi-table Joins

You can join more than two tables.

👉 Example: `users`, `orders`, `payments`

```
SELECT u.name, o.product, p.amount
FROM users u
INNER JOIN orders o ON u.id = o.user_id
INNER JOIN payments p ON o.id = p.order_id;
```

This links **users** → **orders** → **payments**.

7 Self Join

- A table joins with **itself**.
- Use case: employees with managers (both in one table).

👉 Example: `employees`

id	name	manager_id
1	Alice	NULL

```
2 | Bob      | 1
3 | Carol    | 1
4 | David    | 2
```

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

✅ Output:

```
employee | manager
-----|-----
Alice    | NULL
Bob      | Alice
Carol    | Alice
David    | Bob
```

8 Real-world Examples

1. Get all users and their orders (INNER JOIN / LEFT JOIN):

```
SELECT u.name, o.product
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
```

2. Posts with comments (multi-table):

```
SELECT p.title, c.comment
FROM posts p
LEFT JOIN comments c
ON p.id = c.post_id;
```

✅ In summary:

- **INNER JOIN** → only matches
- **LEFT JOIN** → all left + matches
- **RIGHT JOIN** → all right + matches
- **FULL JOIN** → all records from both
- **Self Join** → relate rows in same table

Subqueries & CTEs

1. Subqueries in SELECT, WHERE, FROM

👉 A **subquery** is a query inside another query.
You can use it in different places:

(a) Subquery in SELECT

- Useful for derived values.

```
SELECT
    user_id,
    (SELECT COUNT(*)
     FROM orders o
     WHERE o.user_id = u.user_id) AS total_orders
FROM users u;
```

✅ Explanation:

For each user, we calculate how many orders they placed by running a subquery.

(b) Subquery in WHERE

- Used for filtering rows.

```
SELECT user_id, name
FROM users
WHERE user_id IN (
    SELECT DISTINCT user_id
    FROM orders
    WHERE order_date >= '2025-01-01'
);
```

✓ **Explanation:**

Find users who placed **at least one order in 2025**.

(c) Subquery in FROM (Derived Table)

- You can treat a subquery as a temporary table.

```
SELECT region, AVG(order_total) AS avg_order
FROM (
    SELECT user_id, region, SUM(amount) AS order_total
    FROM orders
    GROUP BY user_id, region
) t
GROUP BY region;
```

✓ **Explanation:**

First, we calculate each user's total order value (inner query).
Then, from that, we calculate **average per region** (outer query).

2. EXISTS vs. IN

Both are used for filtering, but have differences:

IN

Checks if a value exists in a list returned by a subquery.

```
SELECT name
FROM users u
WHERE u.user_id IN (SELECT user_id FROM orders WHERE amount > 100);
```

✅ **Explanation:**

Find users who placed orders greater than 100.

EXISTS

Checks if the subquery returns any rows (more efficient for large datasets).

```
SELECT name
FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.user_id = u.user_id AND o.amount > 100
);
```

✅ **Explanation:**

For each user, check if they have **at least one order above 100**.

- **EXISTS** stops checking as soon as it finds a match → faster.
 - **IN** compares all values → can be slower.
-

3. Common Table Expressions (CTE)

CTEs make queries **cleaner and reusable**.

👉 Syntax:

```
WITH cte_name AS (
    SELECT ...
)
```

```
SELECT * FROM cte_name;
```

Example:

```
WITH user_orders AS (  
    SELECT user_id, SUM(amount) AS total_spent  
    FROM orders  
    GROUP BY user_id  
)  
SELECT u.name, user_orders.total_spent  
FROM users u  
JOIN user_orders ON u.user_id = user_orders.user_id  
WHERE user_orders.total_spent > 1000;
```

✓ Explanation:

1. First, create a CTE `user_orders` to calculate each user's total spending.
2. Then filter those who spent **more than 1000**.

4. Real-World Example: Find users with orders above average value

👉 Problem: Find all users who placed an order **greater than the average order value**.

Solution 1 (Subquery in WHERE)

```
SELECT DISTINCT u.user_id, u.name  
FROM users u  
JOIN orders o ON u.user_id = o.user_id  
WHERE o.amount > (  
    SELECT AVG(amount)  
    FROM orders  
);
```

✓ Steps:

1. Inner query → `SELECT AVG(amount) FROM orders` → finds **average order value**.
 2. Outer query → fetches users whose order amount is **greater than that average**.
-

Solution 2 (Using CTE)

```
WITH avg_value AS (  
    SELECT AVG(amount) AS avg_order_value  
    FROM orders  
)  
SELECT DISTINCT u.user_id, u.name  
FROM users u  
JOIN orders o ON u.user_id = o.user_id, avg_value a  
WHERE o.amount > a.avg_order_value;
```

✓ Steps:

1. CTE `avg_value` → stores the average order value.
 2. Use it in the main query to filter orders above average.
-

💡 Interview Tip:

- If asked: “When do you use `EXISTS` vs `IN`?” → Say:
 - Use `IN` when comparing with a **small static list**.
 - Use `EXISTS` when checking for **correlated rows in another table** (scales better).

What is DCL in MySQL?

DCL (Data Control Language) is used to **control who can do what** in your MySQL database. The main commands are:

- **GRANT** → Give permissions to a user.
 - **REVOKE** → Remove permissions from a user.
-

GRANT

Syntax:

```
GRANT privilege_type
ON database_name.table_name
TO 'username'@'host';
```

Example 1: Grant SELECT on a table

```
-- Create a user
CREATE USER 'webuser'@'localhost' IDENTIFIED BY 'password123';

-- Allow reading data from employees table
GRANT SELECT
ON company_db.employees
TO 'webuser'@'localhost';
```

Example 2: Grant multiple privileges

```
GRANT SELECT, INSERT, UPDATE
ON company_db.employees
TO 'webuser'@'localhost';
```

Example 3: Grant all privileges on a database

```
GRANT ALL PRIVILEGES
ON company_db.*
TO 'webuser'@'localhost';
```

✓ **ALL PRIVILEGES** includes SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, etc.

3 REVOKE

Syntax:

```
REVOKE privilege_type
ON database_name.table_name
FROM 'username'@'host';
```

Example 1: Remove a specific privilege

```
-- Remove UPDATE permission
REVOKE UPDATE
ON company_db.employees
FROM 'webuser'@'localhost';
```

Example 2: Remove all privileges

```
REVOKE ALL PRIVILEGES
ON company_db.*
FROM 'webuser'@'localhost';
```

4 User Roles in MySQL

Roles help manage **groups of privileges** more efficiently.

Step 1: Create a role

```
CREATE ROLE 'webapp_role';
```

Step 2: Grant privileges to the role

```
GRANT SELECT, INSERT, UPDATE
ON company_db.employees
```

```
T0 'webapp_role';
```

Step 3: Assign role to a user

```
GRANT 'webapp_role' TO 'webuser'@'localhost';
```

Step 4: Activate role by default

```
SET DEFAULT ROLE 'webapp_role' FOR 'webuser'@'localhost';
```

Now, `webuser` automatically has the privileges of `webapp_role`.

5 Real-world Example: Web App User

Goal: Allow a web app user to only read/write `employees` table, nothing else.

```
-- Create user for web app
CREATE USER 'webapp'@'localhost' IDENTIFIED BY 'securepass';

-- Grant limited permissions
GRANT SELECT, INSERT, UPDATE
ON company_db.employees
TO 'webapp'@'localhost';
```

✓ The web app **cannot** delete records or access other tables.

TCL (Transaction Control Language) deals with commands that **manage transactions** in a database.



What is a Transaction?

A **transaction** is a set of one or more SQL statements that are executed as a **single logical unit of work**.

- 👉 Either **all statements succeed (COMMIT)**
- 👉 Or **none of them take effect (ROLLBACK)**

This ensures **data consistency and integrity** — the “**all or nothing**” principle.

Main TCL Commands

1 START TRANSACTION / BEGIN

Begins a new transaction.

```
START TRANSACTION;  
-- or  
BEGIN;
```

After this point, all SQL statements are part of that transaction until you either **COMMIT** or **ROLLBACK**.

2 COMMIT

Saves (permanently applies) all changes made in the current transaction.

```
COMMIT;
```

Once you commit, the changes become **permanent** and **visible to others**.

3 ROLLBACK

Undoes all changes made in the current transaction (restores previous state).

```
ROLLBACK;
```

Use this when something goes wrong — e.g., one of your statements fails or a validation check doesn't pass.

4 SAVEPOINT

Creates a **checkpoint** inside a transaction, allowing you to roll back only part of it (not everything).

```
SAVEPOINT save1;  
-- some statements  
ROLLBACK TO save1;
```

You can use multiple SAVEPOINTS in a single transaction.



Real-World Example: Shopping Cart Checkout

Imagine a shopping cart system with two tables:

- CUSTOMER_BALANCE
- ORDER_DETAILS

You want to:

1. Deduct money from the customer's balance.
2. Insert a record into order details.

Both steps must succeed together — otherwise, the database could end up inconsistent (money deducted but no order).

✓ Example in MySQL

```
-- Begin transaction
START TRANSACTION;

-- Step 1: Deduct money from customer balance
UPDATE CUSTOMER_BALANCE
SET BALANCE = BALANCE - 500
WHERE CUSTOMER_ID = 101;

-- Step 2: Insert order record
INSERT INTO ORDER_DETAILS (ORDER_ID, CUSTOMER_ID, AMOUNT)
VALUES (1001, 101, 500);

-- If both succeed
COMMIT;
```

If **something fails**, for example, the balance update failed due to insufficient funds:

```
ROLLBACK;
```

This will undo any partial changes — the money won't be deducted, and the order won't be recorded.

✖ Example with SAVEPOINT

```
START TRANSACTION;

UPDATE CUSTOMER_BALANCE SET BALANCE = BALANCE - 500 WHERE CUSTOMER_ID
= 101;
SAVEPOINT after_balance_deduction;

INSERT INTO ORDER_DETAILS (ORDER_ID, CUSTOMER_ID, AMOUNT)
VALUES (1001, 101, 500);

-- Suppose an error occurs in inserting order
ROLLBACK TO after_balance_deduction;
```

```
-- Continue with other operations or rollback completely  
COMMIT;
```

Summary Table

Command	Purpose	Example
START TRANSACTION / BEGIN	Start new transaction	BEGIN;
COMMIT	Save all changes	COMMIT;
ROLLBACK	Undo all changes	ROLLBACK;
SAVEPOINT	Set partial rollback point	SAVEPOINT s1; ROLLBACK TO s1;